

■ Transformers Complete Guidebook

From Beginner to Intermediate

Hugging Face · Imports · Pipelines · Fine-tuning · LoRA · Deployment · Building from Scratch

June 2026

■ Table of Contents

Ch 1	What is Hugging Face?	3
Ch 2	Installation & Environment Setup	4
Ch 3	Essential Imports — Every Line Explained	5
Ch 4	Tokenizers — Turning Text into Numbers	7
Ch 5	Models — Loading & Understanding Outputs	9
Ch 6	Pipelines — The Fast Track	11
Ch 7	Datasets — Loading & Preprocessing	13
Ch 8	Training & Fine-tuning a Model	15
Ch 9	Evaluation & Metrics	18
Ch 10	Saving, Loading & Sharing Models	19
Ch 11	Building a Transformer from Scratch	21
Ch 12	Quick Reference Cheat Sheet	28
Ch 13	Attention Mechanism — The Math Made Clear	30
Ch 14	Advanced Training Techniques	32
Ch 15	LoRA & PEFT — Fine-tune with 1% of Parameters	34
Ch 16	Text Generation — Decoding Strategies	36
Ch 17	Deploying Your Model	38
Ch 18	Debugging & Common Errors	40
Ch 19	Real-World Project: News Category Classifier	42
Ch 20	Transformer Variants & Your Next Steps	44
Ch 21	Embeddings — The Language of Neural Nets	46
Ch 22	Tokenisation Algorithms — BPE, WordPiece, SP	48
Ch 23	Positional Encoding — Teaching Order to Attention	50
Ch 24	NLP Task Recipes — Copy-Paste Patterns	52
Ch 25	Understanding What Transformers Actually Learn	54
Ch 26	The Mental Model Every Beginner Needs	56
Ch 27	Hyperparameter Reference — Encoder vs Decoder	58
Ch 28	Hyperparameter Tuning — Triage & Diagnosis	64
Ch 29	Data — More Important Than the Model	66
Ch 30	Mistakes I Wish Someone Had Told Me	68
Ch 31	Your First 30 Days: A Practical Action Plan	70

CHAPTER 1

What is Hugging Face?

Hugging Face is the leading open-source platform for Machine Learning. Think of it as the **GitHub of AI models**. It hosts over 500,000 pre-trained models, 100,000+ datasets, and provides the **transformers** Python library that makes using all of them trivially easy.

■ ■ The Hugging Face Ecosystem

The platform consists of several tightly-integrated libraries:

Library	Purpose	Install
transformers	Core — models, tokenizers, trainers	pip install transformers
datasets	Load & process any dataset	pip install datasets
tokenizers	Fast Rust-backed tokenisation	pip install tokenizers
evaluate	Metrics (BLEU, F1, accuracy...)	pip install evaluate
accelerate	GPU/TPU/multi-GPU training	pip install accelerate
peft	Parameter-efficient fine-tuning (LoRA, etc.)	pip install peft
trl	RL from human feedback (RLHF)	pip install trl
diffusers	Image generation models	pip install diffusers

■ All these libraries are from the same org and work seamlessly together. You rarely need to install them all — start with **transformers** and add as needed.

CHAPTER 2

Installation & Environment Setup

■ Step 1 — Create a Virtual Environment

```
# Using venv (standard Python)
python -m venv transformer_env
source transformer_env/bin/activate    # Linux/Mac
transformer_env\Scripts\activate      # Windows

# OR using conda (recommended for GPU work)
conda create -n transformers python=3.11
conda activate transformers
```

■ Step 2 — Install Core Libraries

```
# Minimum installation
pip install transformers datasets

# With PyTorch (CPU)
pip install torch torchvision torchaudio

# With PyTorch (GPU – CUDA 12.1)
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121

# Full ML stack
pip install transformers datasets tokenizers evaluate accelerate
```

■ Step 3 — Verify Installation

```
import torch
import transformers

print("Transformers version:", transformers.__version__)
print("PyTorch version:", torch.__version__)
print("GPU available:", torch.cuda.is_available())
print("GPU name:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "N/A")
```

■ If GPU is False, you are running on CPU. That is fine for learning, but fine-tuning large models will be slow. Use Google Colab (free GPU) for heavy tasks.

CHAPTER 3

Essential Imports — Every Line Explained

This chapter dissects *every* common import you will encounter. Understanding what each class does makes the rest of the library obvious.

■ The Master Import Block

[illegible]

■ What Each Import Actually Does

AutoTokenizer

The smart universal tokenizer loader. Given any model name (e.g. 'bert-base-uncased'), it automatically downloads and loads the correct tokenizer class. You almost never need to import BertTokenizer directly.

```
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
tokens = tokenizer('Hello world!', return_tensors='pt')
# tokens = {'input_ids': tensor, 'attention_mask': tensor, ...}
```

AutoModel

Loads the base model (encoder only, no task head). Returns raw hidden states. Use this when you want raw embeddings or plan to add your own classification head.

```
model = AutoModel.from_pretrained('bert-base-uncased')
outputs = model(**tokens) # Pass tokenizer output directly
embeddings = outputs.last_hidden_state # shape: [batch, seq_len, hidden_size]
```

AutoModelForSequenceClassification

Adds a classification head on top of the base model. Used for tasks like sentiment analysis, spam detection, or any single-label classification.

```
model = AutoModelForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2
)
outputs = model(**tokens)
logits = outputs.logits # shape: [batch_size, num_labels]
```

AutoModelForCausalLM

Loads a causal (left-to-right) language model like GPT-2 or Llama. Used for text generation — the model predicts the next token.

```
from transformers import AutoModelForCausalLM, AutoTokenizer
model = AutoModelForCausalLM.from_pretrained('gpt2')
tokenizer = AutoTokenizer.from_pretrained('gpt2')
inputs = tokenizer('Once upon a time', return_tensors='pt')
output = model.generate(**inputs, max_new_tokens=50)
print(tokenizer.decode(output[0]))
```

AutoModelForSeq2SeqLM

Encoder-Decoder models like T5 or BART. Used for translation, summarisation, and any task where the output is a different sequence from the input.

```
model = AutoModelForSeq2SeqLM.from_pretrained('t5-small')
tokenizer = AutoTokenizer.from_pretrained('t5-small')
inputs = tokenizer('translate English to French: Hello', return_tensors='pt')
output = model.generate(**inputs)
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

Trainer + TrainingArguments

The high-level training loop. `TrainingArguments` holds all hyperparameters. `Trainer` handles the training loop, evaluation, logging, and checkpointing for you.

```
training_args = TrainingArguments(  
    output_dir='./results',  
    num_train_epochs=3,  
    per_device_train_batch_size=16,  
    evaluation_strategy='epoch',  
    save_strategy='epoch',  
    logging_dir='./logs',  
    load_best_model_at_end=True,  
)  
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=tokenized_train,  
    eval_dataset=tokenized_eval,  
    tokenizer=tokenizer,  
    data_collator=DataCollatorWithPadding(tokenizer),  
    compute_metrics=compute_metrics,  
)  
trainer.train()
```

CHAPTER 4

Tokenizers — Turning Text into Numbers

Tokenization is the process of converting raw text into a sequence of integer IDs that a model can process. Every transformer model uses its own vocabulary, so the tokenizer must match the model.

■ How Tokenization Works

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')

# ■■ Basic encoding ████████████████████████████████████████████
text = 'Hello, I love transformers!'
encoded = tokenizer(text)
print(encoded)

# {'input_ids': [101, 7592, 1010, 1045, 2293, 19081, 999, 102],
#   'token_type_ids': [0, 0, 0, 0, 0, 0, 0, 0],
#   'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1]}

# ■■ Decode back to text ████████████████████████████████████████████
decoded = tokenizer.decode(encoded['input_ids'])
print(decoded) # '[CLS] hello, i love transformers! [SEP]'

# ■■ See the actual tokens ████████████████████████████████████████████
tokens = tokenizer.tokenize(text)
print(tokens) # ['hello', ',', ' ', 'i', ' ', 'love', ' ', 'transform', '##ers', '!']
```

■ Key Tokenizer Parameters

[illegible]

■ Special Tokens Explained

Token	BERT ID	Meaning
[CLS]	101	Start of sequence — used for classification
[SEP]	102	Separator between sentences or end of sequence
[PAD]	0	Padding token — ignored by attention mask
[UNK]	100	Unknown token — for words not in vocabulary
[MASK]	103	Masked token — used in masked language modelling

■ *attention_mask* is a tensor of 1s and 0s. A 1 means 'attend to this token', 0 means 'ignore this — it is just padding'. The model uses this to not waste computation on padding positions.

CHAPTER 5

Models — Loading & Understanding Outputs

■ Loading Any Model in 2 Lines

```
from transformers import AutoTokenizer, AutoModel

# 'bert-base-uncased' can be replaced with ANY model on huggingface.co/models
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model      = AutoModel.from_pretrained('bert-base-uncased')

# First time: downloads from Hub (~440 MB for BERT base)
# Subsequent calls: loads from local cache (~/.cache/huggingface/)
```

■ Understanding Model Outputs

```
import torch

text = 'Transformers are amazing!'
inputs = tokenizer(text, return_tensors='pt')

with torch.no_grad(): # no_grad saves memory when not training
    outputs = model(**inputs)

# outputs is a BaseModelOutputWithPoolingAndCrossAttentions object
# Access like a named tuple or dictionary:

last_hidden = outputs.last_hidden_state # shape: [1, seq_len, 768]
# - 1           = batch size
# - seq_len     = number of tokens
# - 768         = hidden dimension (BERT-base)

pooled = outputs.pooler_output # shape: [1, 768]
# The [CLS] token representation – best for classification tasks
```

■■ Model Architecture Reference

Model	Architecture	Best For
BERT	Encoder only	Classification, NER, QA (understanding)

RoBERTa	Encoder only	Same as BERT, stronger training
GPT-2	Decoder only	Text generation, completion
GPT-J/Neo	Decoder only	Text generation (bigger/better)
T5	Encoder-Decoder	Translation, summarisation, QA
BART	Encoder-Decoder	Summarisation, translation
DistilBERT	Encoder only	Faster/smaller BERT — great for production
Llama 2/3	Decoder only	General-purpose generation, chat
Mistral	Decoder only	Efficient generation, long context

■ Memory-Efficient Loading

```
# Half-precision (uses half the VRAM – for GPU users)
model = AutoModel.from_pretrained('bert-base-uncased', torch_dtype=torch.float16)

# 8-bit quantization (needs pip install bitsandbytes)
model = AutoModel.from_pretrained('bert-base-uncased', load_in_8bit=True)

# Move to GPU
device = 'cuda' if torch.cuda.is_available() else 'cpu'
model = model.to(device)
inputs = {k: v.to(device) for k, v in inputs.items()}
```

Pipelines — The Fast Track

Pipelines are the highest-level API in Hugging Face. They bundle tokenizer + model + post-processing into a single callable. Perfect for quick experiments and production APIs.

■ One-Line NLP Tasks


```
# Always specify a model for production (defaults change over time!)
classifier = pipeline(
    task='sentiment-analysis',
    model='distilbert-base-uncased-finetuned-sst-2-english',
    device=0,          # GPU index (use -1 for CPU)
    batch_size=32,     # process 32 samples at once
)

# Batch inference
texts = ['I love this!', 'This is terrible.', 'It was okay.']
results = classifier(texts)
```

■ *Pipelines are production-ready. You can deploy them directly with FastAPI or Gradio. For custom models, pass `model=` and `tokenizer=` arguments.*

CHAPTER 7

Datasets — Loading & Preprocessing

■ Loading Datasets

[illegible]

■ Tokenising a Dataset

[illegible]

CHAPTER 8

Training & Fine-tuning a Model

Fine-tuning means taking a pre-trained model and continuing to train it on your specific task. This is far more effective than training from scratch because the model already understands language — you are just teaching it your specific task.

■ Complete Fine-tuning Example: Sentiment Analysis

Step 1–3: Imports, Data, Tokenization, Model

```
from transformers import (AutoTokenizer,
                          AutoModelForSequenceClassification,
                          Trainer, TrainingArguments, DataCollatorWithPadding)
from datasets import load_dataset
import evaluate, numpy as np

# 1. Load data
raw_dataset = load_dataset('imdb')

# 2. Tokenize
checkpoint = 'distilbert-base-uncased'
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
def tokenize(batch):
    return tokenizer(batch['text'], truncation=True)
tokenized = raw_dataset.map(tokenize, batched=True)

# 3. Load model
model = AutoModelForSequenceClassification.from_pretrained(
    checkpoint, num_labels=2)
```

Step 4–8: Metrics, Training Args, Trainer, Train, Save

```

# 4. Metrics
accuracy = evaluate.load('accuracy')
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return accuracy.compute(predictions=preds, references=labels)

# 5. Training arguments
training_args = TrainingArguments(
    output_dir='./imdb-distilbert',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    warmup_steps=500, weight_decay=0.01,
    evaluation_strategy='epoch',
    save_strategy='epoch',
    load_best_model_at_end=True,
)

# 6. Create Trainer
trainer = Trainer(
    model=model, args=training_args,
    train_dataset=tokenized['train'],
    eval_dataset=tokenized['test'],
    tokenizer=tokenizer,
    data_collator=DataCollatorWithPadding(tokenizer),
    compute_metrics=compute_metrics,
)

# 7. Train!
trainer.train()

# 8. Save
trainer.save_model('./imdb-distilbert-final')
tokenizer.save_pretrained('./imdb-distilbert-final')

```

■ Key TrainingArguments Explained

Argument	Purpose	Typical Value
num_train_epochs	Total training passes over data	3–5
per_device_train_batch_size	Samples per GPU per step	8–32
learning_rate	Step size for optimizer	2e-5 to 5e-5
warmup_steps	Steps to ramp up LR from 0	500
weight_decay	L2 regularisation	0.01
evaluation_strategy	When to evaluate ('epoch' or 'steps')	'epoch'
load_best_model_at_end	Load best checkpoint after training	True
fp16	Use float16 mixed precision (GPU only)	True (GPU)

gradient_accumulation_steps	Simulate larger batch size	2–8
-----------------------------	----------------------------	-----

Evaluation & Metrics

[illegible]

Saving, Loading & Sharing Models

[illegible]

Building a Transformer from Scratch

Now we build the Transformer architecture piece by piece using pure PyTorch. This is the BERT/encoder-style transformer. Each class maps directly to a component in the original 'Attention Is All You Need' paper.

Architecture Overview

Component	Class Name	Purpose
Token Embeddings	TokenEmbedding	Map token IDs → dense vectors
Positional Encoding	PositionalEncoding	Inject position information
Scaled Dot-Product Attn	ScaledDotProductAttention	Core attention mechanism
Multi-Head Attention	MultiHeadAttention	Run H attention heads in parallel
Feed-Forward Layer	FeedForward	Position-wise MLP
Transformer Block	TransformerBlock	Attn + FFN + LayerNorm + Residual
Full Encoder	TransformerEncoder	Stack of N transformer blocks
Classifier Head	TransformerClassifier	Encoder + linear classification head

1 Token + Positional Embeddings


```

import torch
import torch.nn as nn
import math

class TokenEmbedding(nn.Module):
    def __init__(self, vocab_size: int, d_model: int):
        super().__init__()
        # Learnable lookup table: vocab_size rows, d_model cols
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.d_model = d_model

    def forward(self, x):
        # Scale by sqrt(d_model) – from original paper
        return self.embedding(x) * math.sqrt(self.d_model)

class PositionalEncoding(nn.Module):
    '''Adds sinusoidal position information to embeddings.'''
    def __init__(self, d_model: int, max_len: int = 5000, dropout: float = 0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        # Pre-compute PE matrix [max_len, d_model]
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1).float()
        div_term = torch.exp(
            torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
        )
        pe[:, 0::2] = torch.sin(position * div_term) # even indices
        pe[:, 1::2] = torch.cos(position * div_term) # odd indices
        pe = pe.unsqueeze(0) # shape: [1, max_len, d_model]
        self.register_buffer('pe', pe) # saved in state_dict but not trained

    def forward(self, x):
        # x shape: [batch, seq_len, d_model]
        x = x + self.pe[:, :x.size(1)]
        return self.dropout(x)

```

2 ■ Scaled Dot-Product Attention

This is the heart of the Transformer. For every position, we compute how much it should 'attend to' every other position. Q (Query), K (Key), V (Value) are three separate linear projections of the same input.

```

class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout: float = 0.0):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

    def forward(self, Q, K, V, mask=None):
        '''
        Q: [batch, heads, seq, d_k]
        K: [batch, heads, seq, d_k]
        V: [batch, heads, seq, d_v]
        '''
        d_k = Q.size(-1)

        # Step 1: dot product of Q and K → [batch, heads, seq, seq]
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)

        # Step 2: apply mask (for padding or causality)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf'))

        # Step 3: softmax → attention weights (sum to 1 per row)
        attn_weights = torch.softmax(scores, dim=-1)
        attn_weights = self.dropout(attn_weights)

        # Step 4: weighted sum of V
        output = torch.matmul(attn_weights, V)
        return output, attn_weights

```

3 ■ Multi-Head Attention

Instead of one attention operation, we run H independent attention heads, each learning different aspects of the relationships between tokens, then concatenate and project back.

```

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model: int, num_heads: int, dropout: float = 0.1):
        super().__init__()
        assert d_model % num_heads == 0, 'd_model must be divisible by num_heads'

        self.num_heads = num_heads
        self.d_k = d_model // num_heads # dimension per head

        # 4 linear projections: Q, K, V, and output
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

        self.attention = ScaledDotProductAttention(dropout)
        self.dropout = nn.Dropout(dropout)

    def split_heads(self, x, batch_size):
        '''Reshape to [batch, num_heads, seq, d_k]'''
        x = x.view(batch_size, -1, self.num_heads, self.d_k)
        return x.transpose(1, 2)

    def forward(self, Q, K, V, mask=None):
        batch_size = Q.size(0)

        # Project and split into heads
        Q = self.split_heads(self.W_q(Q), batch_size)
        K = self.split_heads(self.W_k(K), batch_size)
        V = self.split_heads(self.W_v(V), batch_size)

        # Attention across all heads simultaneously
        attn_output, _ = self.attention(Q, K, V, mask)

        # Concatenate heads: [batch, seq, d_model]
        attn_output = attn_output.transpose(1, 2).contiguous()
        attn_output = attn_output.view(batch_size, -1, self.num_heads * self.d_k)

        # Final linear projection
        return self.W_o(attn_output)

```

4 ■ Feed-Forward Network

```

class FeedForward(nn.Module):
    '''Position-wise FFN: two linear layers with GELU activation.'''
    def __init__(self, d_model: int, d_ff: int, dropout: float = 0.1):
        super().__init__()
        self.linear1 = nn.Linear(d_model, d_ff)    # expand
        self.linear2 = nn.Linear(d_ff, d_model)    # contract
        self.dropout = nn.Dropout(dropout)
        self.activation = nn.GELU() # smoother than ReLU

    def forward(self, x):
        return self.linear2(self.dropout(self.activation(self.linear1(x))))

```

5 ■ ■ Transformer Block (Encoder Layer)

```

class TransformerBlock(nn.Module):
    '''
    One encoder layer:
        x → LayerNorm → MultiHeadAttention → Residual
          → LayerNorm → FeedForward      → Residual
    Uses Pre-LN (more stable training than original Post-LN).
    '''
    def __init__(self, d_model: int, num_heads: int,
                  d_ff: int, dropout: float = 0.1):
        super().__init__()
        self.attention = MultiHeadAttention(d_model, num_heads, dropout)
        self.ffn        = FeedForward(d_model, d_ff, dropout)
        self.norm1       = nn.LayerNorm(d_model)
        self.norm2       = nn.LayerNorm(d_model)
        self.dropout     = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # ■ ■ Sublayer 1: Self-Attention + Residual ■ ■ ■ ■ ■ ■ ■ ■
        attn_out = self.attention(self.norm1(x),
                                   self.norm1(x),
                                   self.norm1(x), mask)
        x = x + self.dropout(attn_out) # residual connection

        # ■ ■ Sublayer 2: FFN + Residual ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■
        ffn_out = self.ffn(self.norm2(x))
        x = x + self.dropout(ffn_out) # residual connection
        return x

```

6 ■ ■ Full Transformer Encoder + Classifier

```

class TransformerEncoder(nn.Module):
    def __init__(self, vocab_size, d_model=256, num_heads=8,
                  num_layers=4, d_ff=1024, max_len=512, dropout=0.1):
        super().__init__()
        self.token_emb = TokenEmbedding(vocab_size, d_model)
        self.pos_enc = PositionalEncoding(d_model, max_len, dropout)
        self.layers = nn.ModuleList([
            TransformerBlock(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x, mask=None):
        x = self.pos_enc(self.token_emb(x))
        for layer in self.layers:
            x = layer(x, mask)
        return self.norm(x)

class TransformerClassifier(nn.Module):
    def __init__(self, vocab_size, num_classes, d_model=256,
                  num_heads=8, num_layers=4, d_ff=1024,
                  max_len=512, dropout=0.1):
        super().__init__()
        self.encoder = TransformerEncoder(
            vocab_size, d_model, num_heads,
            num_layers, d_ff, max_len, dropout)
        self.classifier = nn.Linear(d_model, num_classes)
        self.dropout = nn.Dropout(dropout)

    def forward(self, input_ids, attention_mask=None):
        # Build padding mask from attention_mask
        mask = None
        if attention_mask is not None:
            # [batch, 1, 1, seq] for broadcasting
            mask = attention_mask.unsqueeze(1).unsqueeze(2)

        encoded = self.encoder(input_ids, mask) # [batch, seq, d_model]
        cls_repr = encoded[:, 0, :] # use [CLS] token
        return self.classifier(self.dropout(cls_repr))

```

7 ■ ■ Instantiate & Verify


```

for epoch in range(NUM_EPOCHS):
    model.train()
    total_loss = 0
    for batch in train_loader:
        input_ids = batch['input_ids'].to(device)
        attn_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        optimizer.zero_grad()
        logits = model(input_ids, attn_mask)
        loss = criterion(logits, labels)
        loss.backward()
        # Gradient clipping prevents exploding gradients
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        total_loss += loss.item()

    scheduler.step()
    print(f'Epoch {epoch+1} Loss: {total_loss/len(train_loader):.4f}')

```

Evaluation Loop

```

model.eval()
correct = total = 0
with torch.no_grad():
    for batch in val_loader:
        input_ids = batch['input_ids'].to(device)
        attn_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)
        preds = model(input_ids, attn_mask).argmax(dim=-1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
print(f' Val Accuracy: {correct/total:.4f}')

```

■ Your from-scratch model won't beat BERT because BERT was trained on 3.3 billion words. But building it teaches you exactly what each component does — essential knowledge for research and debugging. Combine this understanding with HF's pre-trained models for the best of both worlds.

CHAPTER 12

Quick Reference Cheat Sheet

■ One-Liners

```
# Load tokenizer + model
tok = AutoTokenizer.from_pretrained(MODEL); m = AutoModel.from_pretrained(MODEL)

# Tokenise a batch
enc = tok(texts, padding=True, truncation=True, max_length=128, return_tensors='pt')

# Inference
with torch.no_grad(): out = m(**enc)

# Classify
preds = out.logits.argmax(dim=-1)

# Run a pipeline
pipe = pipeline('sentiment-analysis'); pipe('I love this!')

# Load HF dataset
ds = load_dataset('imdb')

# Save / load model
m.save_pretrained('./dir'); m = AutoModel.from_pretrained('./dir')

# Push to Hub
m.push_to_hub('username/model-name')

# Count parameters
sum(p.numel() for p in m.parameters() if p.requires_grad)
```

■ Task → Model Reference

Task	Recommended Model	HF Task Tag
Text classification	distilbert-base-uncased	AutoModelForSequenceClassification
Named Entity Recognition	dslim/bert-base-NER	AutoModelForTokenClassification
Question Answering	deepset/roberta-base-squad2	AutoModelForQuestionAnswering
Text Generation	gpt2, mistralai/Mistral-7B	AutoModelForCausalLM
Summarisation	facebook/bart-large-cnn	AutoModelForSeq2SeqLM
Translation	Helsinki-NLP/opus-mt-en-de	AutoModelForSeq2SeqLM

Sentence Embeddings	sentence-transformers/all-MiniLM	AutoModel
Zero-Shot Classification	facebook/bart-large-mnli	pipeline('zero-shot-classification')
Fill-Mask	bert-base-uncased	AutoModelForMaskedLM
Image Classification	google/vit-base-patch16-224	AutoModelForImageClassification

■ Essential Resources

- **Hugging Face Hub** — huggingface.co/models
- **Transformers Docs** — huggingface.co/docs/transformers
- **Datasets Docs** — huggingface.co/docs/datasets
- **Course (Free)** — huggingface.co/learn/nlp-course
- **Spaces (Deploy apps)** — huggingface.co/spaces
- **Papers with Code** — paperswithcode.com
- **PyTorch Docs** — pytorch.org/docs

CHAPTER 13

Attention Mechanism — The Math Made Clear

You have seen the code — now let us understand precisely what the math is doing. Every transformer operation maps directly to a matrix multiplication you can visualise.

■ The Three Matrices: Q, K, V

Given an input sequence X of shape $[\text{seq_len} \times \text{d_model}]$, we create three projections using learned weight matrices. This is the ONLY place the model 'learns' in attention — everything else is deterministic math.

```
# Input X: shape [batch, seq_len, d_model] e.g. [1, 5, 512]
# Weight matrices: learned during training
W_Q = nn.Linear(d_model, d_k) # [512, 64]
W_K = nn.Linear(d_model, d_k) # [512, 64]
W_V = nn.Linear(d_model, d_v) # [512, 64]

Q = W_Q(X) # Queries: 'what am I looking for?' [batch, seq, 64]
K = W_K(X) # Keys: 'what do I contain?' [batch, seq, 64]
V = W_V(X) # Values: 'what do I actually give?' [batch, seq, 64]
```

■ Step-by-Step Attention Computation

Step	Formula	Shape	What It Does
1. Raw scores	$Q \times K^T$	$[\text{seq}, \text{seq}]$	Dot product: how similar is each Q to each K?
2. Scale	$/ \sqrt{d_k}$	$[\text{seq}, \text{seq}]$	Prevents softmax saturation for large d_k
3. Mask (opt.)	fill -inf where mask=1	$[\text{seq}, \text{seq}]$	Block padding or future tokens
4. Softmax	softmax(row-wise)	$[\text{seq}, \text{seq}]$	Normalise to probabilities (sum=1 per row)
5. Weighted sum	$\text{Attn} \times V$	$[\text{seq}, d_v]$	Each output = weighted combination of Values

■ Visualising What Attention Learns

```
# Extract attention weights from a HuggingFace model
from transformers import AutoTokenizer, AutoModel
import torch, matplotlib.pyplot as plt, seaborn as sns

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model      = AutoModel.from_pretrained('bert-base-uncased',
                                       output_attentions=True)

text      = 'The cat sat on the mat'
inputs    = tokenizer(text, return_tensors='pt')
tokens    = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])

with torch.no_grad():
    outputs = model(**inputs)

# outputs.attentions: tuple of (num_layers) tensors
# Each tensor shape: [batch, num_heads, seq_len, seq_len]
attn = outputs.attentions
print(f'Num layers: {len(attn)}')          # 12 for BERT-base
print(f'Shape per layer: {attn[0].shape}') # [1, 12, 8, 8]

# Plot layer 0, head 0
layer, head = 0, 0
attn_matrix = attn[layer][0, head].numpy()

plt.figure(figsize=(8, 6))
sns.heatmap(attn_matrix, xticklabels=tokens, yticklabels=tokens,
            cmap='Blues', annot=True, fmt='.2f')
plt.title(f'Attention weights – Layer {layer}, Head {head}')
plt.tight_layout()
plt.savefig('attention_heatmap.png', dpi=150)
```

■ *Different heads specialise in different patterns: some track syntactic dependencies (subject-verb), some co-reference (pronoun → noun), some positional (nearby tokens). This is why multi-head attention is powerful — each head learns a different 'lens'.*

■ ■ Self-Attention vs Cross-Attention vs Causal Attention

Type	Where Used	Q comes from	K, V come from	Mask
Self-Attention	BERT encoder	Same sequence	Same sequence	Padding only
Causal (Masked)	GPT decoder	Same sequence	Same sequence	Future tokens blocked
Cross-Attention	T5/BART decoder	Decoder	Encoder output	Padding only

Advanced Training Techniques

■ Mixed Precision Training (fp16 / bf16)

Full precision (fp32) uses 4 bytes per parameter. Half precision (fp16/bf16) uses 2 bytes, roughly doubling the batch size you can fit in GPU memory and speeding up training by 2–3x on modern hardware with no meaningful accuracy loss.

```
# Method 1: via TrainingArguments (easiest)
training_args = TrainingArguments(
    ...
    fp16=True,    # NVIDIA GPUs (V100, T4, A100)
    # bf16=True, # Ampere+ GPUs (A100, H100) – more numerically stable
)

# Method 2: manual with torch.autocast
from torch.cuda.amp import autocast, GradScaler
scaler = GradScaler()

for batch in train_loader:
    optimizer.zero_grad()
    with autocast():
        outputs = model(**batch)
        loss = outputs.loss
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```

■ Learning Rate Schedules

The learning rate schedule determines how the LR changes during training. A warmup phase prevents early instability; decay prevents late-stage overshooting.

```

from transformers import get_linear_schedule_with_warmup
from transformers import get_cosine_schedule_with_warmup
from transformers import get_constant_schedule_with_warmup

# Total training steps
num_steps = len(train_loader) * num_epochs
warmup = int(0.1 * num_steps) # 10% warmup

# Option A: Linear decay (default for BERT fine-tuning)
scheduler = get_linear_schedule_with_warmup(
    optimizer, num_warmup_steps=warmup, num_training_steps=num_steps
)

# Option B: Cosine decay (smoother, often better)
scheduler = get_cosine_schedule_with_warmup(
    optimizer, num_warmup_steps=warmup, num_training_steps=num_steps
)

# Inside training loop:
loss.backward()
optimizer.step()
scheduler.step() # call EVERY step, not every epoch
optimizer.zero_grad()

```

■ Gradient Accumulation — Simulate Large Batches

If your GPU cannot fit a batch of 32, you can accumulate gradients over 4 steps of batch size 8 to effectively get the same result. The weights only update every N steps.

```

# Via TrainingArguments
training_args = TrainingArguments(
    ...
    per_device_train_batch_size=8,
    gradient_accumulation_steps=4, # effective batch = 8 × 4 = 32
)

# Manual implementation
ACCUMULATION_STEPS = 4
optimizer.zero_grad()

for step, batch in enumerate(train_loader):
    outputs = model(**batch)
    loss = outputs.loss / ACCUMULATION_STEPS # normalize!
    loss.backward()

    if (step + 1) % ACCUMULATION_STEPS == 0:
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()

```

■ Early Stopping & Best Model Checkpointing

```
from transformers import EarlyStoppingCallback

trainer = Trainer(
    ...
    callbacks=[EarlyStoppingCallback(early_stopping_patience=3)]
    # Stops if eval metric doesn't improve for 3 consecutive evals
)

# TrainingArguments must have:
training_args = TrainingArguments(
    ...
    evaluation_strategy='epoch',
    save_strategy='epoch',
    load_best_model_at_end=True,
    metric_for_best_model='eval_f1',  # your metric name
    greater_is_better=True,
)
```

■ Experiment Tracking with Weights & Biases

```
# pip install wandb
import wandb
wandb.login()  # enter your API key once

training_args = TrainingArguments(
    ...
    report_to='wandb',
    run_name='bert-imdb-exp1',
    logging_steps=50,
)

# wandb automatically logs: loss, lr, gradient norms,
# eval metrics, system metrics (GPU util, memory)
# View at: wandb.ai/your-username/your-project
```

LoRA & PEFT — Fine-tune with 1% of Parameters

Fine-tuning a 7B parameter LLM updates all 7 billion weights — requiring 80+ GB of GPU RAM. Parameter-Efficient Fine-Tuning (PEFT) methods like LoRA (Low-Rank Adaptation) freeze the original model and inject tiny trainable matrices, reducing trainable params by 99% while achieving nearly identical performance.

■ How LoRA Works

For a weight matrix W of shape $[d \times k]$, instead of updating W directly, LoRA adds two small matrices: A of shape $[d \times r]$ and B of shape $[r \times k]$, where $r \ll d$. The output becomes: $W \cdot x + (B \cdot A) \cdot x \times \text{scaling_factor}$. Only A and B are trained. With $r=8$ on a 768×768 matrix: $768 \times 768 = 589,824$ params $\rightarrow 2 \times (768 \times 8) = 12,288$ params. That is 48x fewer.

```
# pip install peft
from peft import LoraConfig, get_peft_model, TaskType
from transformers import AutoModelForSequenceClassification

# 1. Load base model
model = AutoModelForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2
)

# 2. Define LoRA config
lora_config = LoraConfig(
    task_type=TaskType.SEQ_CLS,
    r=8,                      # rank — higher = more capacity
    lora_alpha=32,            # scaling factor (alpha/r controls magnitude)
    target_modules=['query', 'value'], # which weight matrices to adapt
    lora_dropout=0.1,
    bias='none',
)

# 3. Wrap model with LoRA
model = get_peft_model(model, lora_config)

# 4. See the difference
model.print_trainable_parameters()
# trainable params: 296,450 || all params: 109,779,202
# trainable%: 0.27% <-- only 0.27% of weights are updated!
```

■ QLoRA — 4-bit Quantised LoRA (for LLMs on consumer GPUs)

QLoRA combines 4-bit quantization (NF4) with LoRA, enabling fine-tuning of 7B+ models on a single 24 GB GPU. This is how most people fine-tune Llama, Mistral, etc.

```
# pip install peft bitsandbytes transformers accelerate
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
import torch

# 1. 4-bit quantization config
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4',          # NormalFloat4
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,
)

# 2. Load model in 4-bit
model = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Llama-2-7b-hf',
    quantization_config=bnb_config,
    device_map='auto',
)

# 3. Prepare for training
model = prepare_model_for_kbit_training(model)

# 4. Add LoRA adapters on top
lora_config = LoraConfig(
    r=16, lora_alpha=64,
    target_modules=['q_proj', 'v_proj', 'k_proj', 'o_proj'],
    lora_dropout=0.05, bias='none',
    task_type='CAUSAL_LM',
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# trainable params: 39M / 3.5B total — only 1.1%!
```

■ Saving & Loading PEFT Adapters


```
# Save only the adapter (tiny – ~20 MB vs 14 GB for full model)
model.save_pretrained('./llama2-lora-adapter')

# Load adapter on top of base model later
from peft import PeftModel
base_model = AutoModelForCausalLM.from_pretrained('meta-llama/Llama-2-7b-hf',
                                                    quantization_config=bnb_config)
model = PeftModel.from_pretrained(base_model, './llama2-lora-adapter')

# Merge adapter weights into model (for faster inference)
model = model.merge_and_unload()
```

■ LoRA rank (r) is the most important hyperparameter. $r=4$ is tiny/fast, $r=16$ is a good default, $r=64$ approaches full fine-tuning. Start with $r=16$, $\text{lora_alpha}=32$ ($\text{alpha}=2r$ is a common heuristic).

Text Generation — Decoding Strategies

When a model generates text, it produces a probability distribution over the vocabulary at each step. The decoding strategy determines HOW we pick the next token from that distribution. This choice dramatically affects output quality, diversity, and coherence.

■ All Decoding Strategies Compared

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch
model = AutoModelForCausalLM.from_pretrained('gpt2')
tokenizer = AutoTokenizer.from_pretrained('gpt2')
inputs = tokenizer('The future of AI is', return_tensors='pt')

# 1. Greedy – always pick the most probable next token
greedy = model.generate(**inputs, max_new_tokens=50)

# 2. Beam Search – keep top-k candidate sequences in parallel
beam = model.generate(**inputs, max_new_tokens=50,
                      num_beams=5, early_stopping=True, no_repeat_ngram_size=2)

# 3. Sampling – randomly sample from the full distribution
sample = model.generate(**inputs, max_new_tokens=50,
                        do_sample=True, temperature=0.8)

# 4. Top-k – sample from the top-k most likely tokens only
topk = model.generate(**inputs, max_new_tokens=50,
                      do_sample=True, top_k=50, temperature=0.9)

# 5. Top-p (Nucleus) – sample tokens covering 92% probability mass
topp = model.generate(**inputs, max_new_tokens=100,
                      do_sample=True, top_p=0.92, temperature=0.85,
                      repetition_penalty=1.2)

# 6. Contrastive Search – best coherence + diversity balance
contrastive = model.generate(**inputs, max_new_tokens=100,
                             penalty_alpha=0.6, top_k=4)

# Decode all outputs
for name, out in [ ('Greedy', greedy), ('Beam', beam), ('Sample', sample),
                  ('Top-k', topk), ('Top-p', topp), ('Contrastive', contrastive) ]:
    print(f'{name}: {tokenizer.decode(out[0], skip_special_tokens=True)}')
```

■ Temperature — Intuition

Temperature	Effect	Best For
0.1 – 0.3	Very focused, near-deterministic	Factual Q&A, code completion
0.6 – 0.8	Balanced coherence and variety	General conversation, summaries
1.0	Unmodified model distribution	Baseline experiments
1.2 – 1.5	Creative, unpredictable	Brainstorming, creative writing
> 2.0	Mostly noise	Rarely useful

■ Chat Formatting (for Instruction-Tuned Models)

Instruction-tuned models (ChatGPT-style) expect a specific prompt format. Using the wrong format gives poor results. HuggingFace's `apply_chat_template` handles this automatically.

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

checkpoint = 'mistralai/Mistral-7B-Instruct-v0.2'
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForCausalLM.from_pretrained(
    checkpoint, torch_dtype=torch.float16, device_map='auto')

# Structured conversation
messages = [{'role': 'user', 'content': 'Explain transformers in 2 sentences.'}]

# apply_chat_template formats for this model's expected format
# Mistral uses: [INST] ... [/INST]
input_ids = tokenizer.apply_chat_template(
    messages, return_tensors='pt',
    add_generation_prompt=True, # adds assistant turn trigger
).to(model.device)

output = model.generate(
    input_ids, max_new_tokens=200, do_sample=True,
    temperature=0.7, top_p=0.9,
)
new_tokens = output[0][input_ids.shape[1]:]
print(tokenizer.decode(new_tokens, skip_special_tokens=True))
```

Deploying Your Model

Training is only half the journey. Here we cover three ways to serve your model: a FastAPI REST endpoint, a Gradio web demo, and Hugging Face Spaces (free hosting).

■ Option 1: FastAPI REST API

```
# pip install fastapi uvicorn
from fastapi import FastAPI
from pydantic import BaseModel
from transformers import pipeline
import uvicorn

# Load model once at startup
classifier = pipeline(
    'sentiment-analysis',
    model='distilbert-base-uncased-finetuned-sst-2-english',
    device=0 if torch.cuda.is_available() else -1,
)

app = FastAPI(title='Sentiment API')

class TextInput(BaseModel):
    text: str

class BatchInput(BaseModel):
    texts: list[str]

@app.get('/')
def root(): return {'status': 'running'}

@app.post('/predict')
def predict(body: TextInput):
    result = classifier(body.text)[0]
    return {'label': result['label'], 'score': round(result['score'], 4)}

@app.post('/predict_batch')
def predict_batch(body: BatchInput):
    results = classifier(body.texts)
    return {'predictions': results}

# Run with: uvicorn app:app --host 0.0.0.0 --port 8000
# Test:      curl -X POST http://localhost:8000/predict
#           -H 'Content-Type: application/json'
#           -d '{"text": "I love this product!"}'
```

■ Option 2: Gradio Web Demo (3 lines)

```
# pip install gradio
import gradio as gr
from transformers import pipeline

classifier = pipeline('sentiment-analysis')

def predict(text):
    result = classifier(text)[0]
    return f"{result['label']} ({result['score']:.1%} confidence)"

# Launch a web UI instantly
demo = gr.Interface(
    fn=predict,
    inputs=gr.Textbox(label='Enter text', placeholder='Type something...'),
    outputs=gr.Text(label='Sentiment'),
    title='Sentiment Analyser',
    description='Powered by DistilBERT',
    examples=[['I love this!'], ['This is terrible.'], ['It was okay.']],
)
demo.launch(share=True) # share=True creates a public link
```

■■ Option 3: Deploy to Hugging Face Spaces (Free)

Spaces is free hosting for ML demos. Create a Space on huggingface.co/new-space, choose Gradio, and push your code. The app.py file is all you need.

```

# app.py – the ONLY file needed for a Space
import gradio as gr
from transformers import pipeline

# Spaces auto-installs from requirements.txt
pipe = pipeline('text-generation', model='gpt2')

def generate(prompt, max_tokens=100, temperature=0.8):
    out = pipe(prompt, max_new_tokens=max_tokens,
                temperature=temperature, do_sample=True)
    return out[0]['generated_text']

gr.Interface(
    fn=generate,
    inputs=[
        gr.Textbox(label='Prompt'),
        gr.Slider(50, 200, value=100, label='Max tokens'),
        gr.Slider(0.1, 2.0, value=0.8, step=0.1, label='Temperature'),
    ],
    outputs=gr.Textbox(label='Generated text'),
    title='GPT-2 Text Generator',
).launch()

# requirements.txt (place in same folder):
# transformers
# torch
# gradio

```

■ For production, use `model.half()` (fp16), `torch.compile()` (PyTorch 2.0+), and batch your requests. A DistilBERT classifier on a T4 GPU can handle ~2000 requests/second with batching.

CHAPTER 18

Debugging & Common Errors

■ The Most Common Errors & Fixes

■ CUDA out of memory

```
# Error: RuntimeError: CUDA out of memory. Tried to allocate X GiB
```

■ Fix:

```
# Fix 1: Reduce batch size
per_device_train_batch_size = 4    # reduce from 16

# Fix 2: Enable gradient checkpointing
model.gradient_checkpointing_enable()

# Fix 3: Use fp16
training_args = TrainingArguments(..., fp16=True)

# Fix 4: Clear cache between runs
torch.cuda.empty_cache()
import gc; gc.collect()

# Fix 5: Use gradient accumulation to simulate large batch
gradient_accumulation_steps = 4
```

■ Input IDs too long / truncation warning

```
# Error: Token indices sequence length is longer than the specified maximum
```

■ Fix:

```
# Always pass truncation=True
tokenizer(text, truncation=True, max_length=512)

# Or set model_max_length on tokenizer
tokenizer.model_max_length = 512
```

■ Label column missing / wrong name

```
# Error: KeyError: 'labels' or ValueError: The model did not return a loss
```

■ Fix:

```
# Trainer expects column named 'labels', not 'label'
dataset = dataset.rename_column('label', 'labels')

# Or handle it in tokenize function:
def tokenize(batch):
    enc = tokenizer(batch['text'], truncation=True)
    enc['labels'] = batch['label']
    return enc
```

■ Model not on same device as input

```
# Error: RuntimeError: Expected all tensors to be on same device
```

■ Fix:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)

# Move ALL inputs to same device
inputs = {k: v.to(device) for k, v in inputs.items()}

# Or use device_map='auto' for large models
model = AutoModel.from_pretrained(name, device_map='auto')
```

■ NaN loss during training

```
# Error: Loss becomes NaN after a few steps
```

■ Fix:

```
# Cause 1: LR too high – lower it
learning_rate = 1e-5 # try lower values

# Cause 2: No gradient clipping
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

# Cause 3: fp16 overflow – switch to bf16 or add loss scaling
training_args = TrainingArguments(..., bf16=True)

# Cause 4: Bad data (infinities) – check your dataset
import pandas as pd
df[df['text'].isna()] # check for NaN values
```

■ Debugging Tools

[illegible]

CHAPTER 19

Real-World Project: News Category Classifier

Let us put everything together in one cohesive project: a news article classifier that categorises articles into topics (sports, politics, tech, etc.) using DistilBERT. This is a real pattern used in production systems.

■ Project Structure

```
news-classifier/  
■■■ train.py           # training pipeline  
■■■ inference.py       # load model + predict  
■■■ app.py             # Gradio web demo  
■■■ requirements.txt  
■■■ results/           # saved model checkpoints
```

■■ train.py — Full Training Script

Part 1: Imports, Data, Tokenize, Model

```
# train.py  
from transformers import (AutoTokenizer,  
                          AutoModelForSequenceClassification,  
                          Trainer, TrainingArguments, DataCollatorWithPadding)  
from datasets import load_dataset  
import evaluate, numpy as np  
  
CHECKPOINT = 'distilbert-base-uncased'  
OUTPUT_DIR = './results/news-classifier'  
  
dataset = load_dataset('ag_news') # World/Sports/Business/Sci-Tech  
tokenizer = AutoTokenizer.from_pretrained(CHECKPOINT)  
label2id = {0:'World', 1:'Sports', 2:'Business', 3:'Sci/Tech'}  
id2label = {v: k for k, v in label2id.items()}  
  
def tokenize(batch):  
    return tokenizer(batch['text'], truncation=True, max_length=256)  
tokenized = dataset.map(tokenize, batched=True, remove_columns=['text'])  
tokenized = tokenized.rename_column('label', 'labels')  
  
model = AutoModelForSequenceClassification.from_pretrained(  
    CHECKPOINT, num_labels=4, id2label=id2label, label2id=label2id)
```

Part 2: Metrics, Args, Train, Save

```
accuracy = evaluate.load('accuracy')
def metrics(eval_pred):
    preds = np.argmax(eval_pred.predictions, axis=-1)
    return accuracy.compute(predictions=preds,
                             references=eval_pred.label_ids)

args = TrainingArguments(
    output_dir=OUTPUT_DIR, num_train_epochs=3,
    per_device_train_batch_size=32, learning_rate=2e-5,
    warmup_ratio=0.06, weight_decay=0.01,
    evaluation_strategy='epoch', save_strategy='epoch',
    load_best_model_at_end=True, fp16=True, report_to='none',
)
trainer = Trainer(
    model=model, args=args,
    train_dataset=tokenized['train'],
    eval_dataset=tokenized['test'],
    tokenizer=tokenizer,
    data_collator=DataCollatorWithPadding(tokenizer),
    compute_metrics=metrics,
)
trainer.train()
trainer.save_model(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)
```

■ inference.py — Production Inference

```

# inference.py
from transformers import pipeline
import torch

class NewsClassifier:
    def __init__(self, model_path='./results/news-classifier'):
        self.pipe = pipeline(
            'text-classification',
            model=model_path,
            device=0 if torch.cuda.is_available() else -1,
        )

    def predict(self, text: str) -> dict:
        result = self.pipe(text, truncation=True)[0]
        return {'category': result['label'], 'confidence': result['score']}

    def predict_batch(self, texts: list[str]) -> list[dict]:
        results = self.pipe(texts, truncation=True, batch_size=32)
        return [{'category': r['label'], 'confidence': r['score']}
                for r in results]

# Usage
if __name__ == '__main__':
    clf = NewsClassifier()
    article = '''
        Apple announced record quarterly earnings driven by iPhone sales
        and services growth, beating analyst expectations by 12 percent.
    '''
    print(clf.predict(article))
    # {'category': 'Business', 'confidence': 0.9871}

```

■ Expected accuracy: ~94% on the AG News test set after 3 epochs. Training time: ~8 minutes on a T4 GPU (Google Colab free tier). The same pattern works for any multi-class text classification task — just swap the dataset and num_labels.

CHAPTER 20

Transformer Variants & Your Next Steps

■ ■ The Transformer Family Tree

Model	Year	Key Innovation	Use Today If...
Transformer	2017	Original Attn Is All You Need paper	You want to understand history
BERT	2018	Bidirectional masked language modelling	Classification, NER, QA tasks
GPT-2	2019	Large-scale causal language modelling	Lightweight text generation
RoBERTa	2019	Better BERT training recipe	BERT tasks — usually outperforms BERT
T5	2020	Unify all NLP as text-to-text	Translation, summarisation, QA
DistilBERT	2020	Knowledge distillation — 40% smaller BERT	Production: speed matters
GPT-3	2020	Scale + few-shot prompting	Reference only (closed)
DeBERTa	2021	Disentangled attention (SOTA on GLUE)	Max accuracy on classification
FLAN-T5	2022	Instruction-tuned T5	Open instruction following
LLaMA 2	2023	Open-weight efficient LLM	Fine-tune your own chatbot
Mistral 7B	2023	Sliding window attention, efficient	Best open 7B model (2023)
Llama 3	2024	Improved tokenizer, 128k context	State-of-art open model

■ ■ Your Learning Roadmap

Week 1–2 — Beginner

Run pipelines, tokenize text, load models. Complete the free HF NLP Course (huggingface.co/learn/nlp-course).

Week 3–4 — Beginner+

Fine-tune DistilBERT on a classification dataset. Experiment with different datasets from the Hub.

Month 2 — Intermediate

Learn LoRA/PEFT. Fine-tune a 7B LLM with QLoRA on Colab. Build and deploy a Gradio app.

Month 3 — Intermediate+

Implement a transformer from scratch. Read the original Attention paper (arxiv 1706.03762). Study BERT paper.

Month 4–6 — Advanced

Custom training loops. Multi-GPU with Accelerate. Experiment tracking with W&B.; RLHF with TRL library.

Month 6+ — Research

Read recent papers on PaperswithCode. Reproduce results. Contribute to HF or write your own models.

■ Must-Read Papers (in Order)

- **Attention Is All You Need (2017)** — The original Transformer paper. arxiv.org/abs/1706.03762
- **BERT (2018)** — Bidirectional pre-training. arxiv.org/abs/1810.04805
- **Language Models are Few-Shot Learners — GPT-3 (2020)** — Scale + prompting. arxiv.org/abs/2005.14165
- **LoRA (2021)** — Efficient fine-tuning. arxiv.org/abs/2106.09685
- **Training language models from human feedback — RLHF (2022)** — Alignment. arxiv.org/abs/2203.02155
- **LLaMA 2 (2023)** — Best open-weight LLM paper. arxiv.org/abs/2307.09288

Embeddings — The Language of Neural Nets

An embedding is a dense vector of floating-point numbers that represents a token, word, sentence, or document. The magic of transformers is that these vectors are **learned** — similar concepts end up close together in vector space, and relationships like $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ emerge naturally from training.

■ Why Vectors, Not Integers?

Imagine assigning each word an ID: 'cat'=1, 'dog'=2, 'car'=500. These integers carry no meaning — the model has no way to know that 1 and 2 are related (both animals) while 1 and 500 are not. An embedding solves this: each ID maps to a 768-dimensional vector, and training adjusts these vectors so semantically similar words become geometrically close.

```
import torch, torch.nn as nn

vocab_size = 10000 # 10k unique tokens
d_model     = 768  # 768-dimensional vectors (BERT-base)

embedding = nn.Embedding(vocab_size, d_model)
# This is a matrix: [10000, 768] = 7.68 million learnable parameters

token_ids = torch.tensor([101, 7592, 2088, 102]) # [CLS] hello world [SEP]
vectors    = embedding(token_ids) # shape: [4, 768]
print(vectors.shape) # torch.Size([4, 768])

# After training:
# cos_sim('king', 'queen') ≈ 0.85 (very similar)
# cos_sim('king', 'banana') ≈ 0.05 (unrelated)
```

■ Measuring Similarity: Cosine Similarity

Two vectors are 'similar' if they point in the same direction, regardless of magnitude. Cosine similarity measures this angle — 1.0 means identical direction, 0 means perpendicular (unrelated), -1 means opposite. It is the standard metric for comparing embeddings.

```

import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model = AutoModel.from_pretrained('bert-base-uncased')

def get_embedding(text):
    inputs = tokenizer(text, return_tensors='pt')
    with torch.no_grad():
        outputs = model(**inputs)
    # Mean pooling over all token embeddings
    return outputs.last_hidden_state.mean(dim=1) # [1, 768]

e1 = get_embedding('The king ruled the kingdom')
e2 = get_embedding('The queen governed the realm')
e3 = get_embedding('I like to eat pizza')

print(f'King vs Queen: {F.cosine_similarity(e1, e2).item():.3f}') # ~0.92
print(f'King vs Pizza: {F.cosine_similarity(e1, e3).item():.3f}') # ~0.45

```

■ Sentence Transformers — Purpose-Built Sentence Embeddings

Raw BERT embeddings are good for word-level tasks but suboptimal for whole-sentence similarity. Sentence Transformers (SBERT) are models fine-tuned specifically to produce high-quality sentence embeddings using contrastive learning on sentence pairs. They power semantic search, clustering, and RAG systems.

```

# pip install sentence-transformers
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer('all-MiniLM-L6-v2') # tiny + fast

sentences = [
    'A dog is playing fetch in the park',
    'A puppy is chasing a ball outside',
    'The stock market fell sharply today',
]
embeddings = model.encode(sentences, convert_to_tensor=True)
# shape: [3, 384] (MiniLM uses 384 dims)

cosine_scores = util.cos_sim(embeddings, embeddings)
# [[1.00, 0.89, 0.12],
#  [0.89, 1.00, 0.14],
#  [0.12, 0.14, 1.00]]

# Semantic search: find most similar sentence to a query
query = 'What is my dog doing?'
q_emb = model.encode(query, convert_to_tensor=True)
hits = util.semantic_search(q_emb, embeddings, top_k=2)
print(hits)

```


■ Semantic Search with FAISS (Production Scale)

When you have millions of documents, comparing your query against every one is too slow. FAISS (Facebook AI Similarity Search) builds an approximate nearest-neighbour index that finds the most similar vectors in milliseconds even across 100 million entries. This is the engine behind most production search and RAG systems.

```
# pip install faiss-cpu
import faiss, numpy as np
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')
dim = 384

documents = ['doc 1 text', 'doc 2 text', ...]
embeddings = model.encode(documents).astype(np.float32)
faiss.normalize_L2(embeddings)

index = faiss.IndexFlatIP(dim) # Inner Product after normalisation = cosine
index.add(embeddings)

query = 'What causes inflation?'
q_emb = model.encode([query]).astype(np.float32)
faiss.normalize_L2(q_emb)
distances, indices = index.search(q_emb, k=5) # top 5 results

for i, idx in enumerate(indices[0]):
    print(f'#{i+1} score={distances[0][i]:.3f}: {documents[idx][:80]}')
```

■ *Sentence Transformers + FAISS is the backbone of most RAG (Retrieval-Augmented Generation) systems. Encode your documents once, store in FAISS, then at query time retrieve the top-k most relevant passages and feed them to an LLM as context.*

Tokenisation Algorithms — BPE, WordPiece & SentencePiece

Not all tokenizers work the same way. Different models use different algorithms to split text into tokens. Understanding WHY matters: it explains why 'unhappy' becomes ['un', '##happy'] in BERT but ['un', 'happy'] in GPT-2, and why some models handle rare words or new languages better than others.

■ The Core Problem: Vocabulary vs Coverage

A character-level tokenizer can represent any word but creates very long sequences. A word-level tokenizer keeps sequences short but cannot handle new words (the out-of-vocabulary problem). Subword tokenization is the middle ground: split rare words into known pieces, keep common words whole. All modern transformers use some form of subword tokenization.

Method 1: Byte-Pair Encoding (BPE) — GPT-2, GPT-3, Llama

BPE starts with individual characters, then iteratively merges the most frequent adjacent pair of symbols until the vocabulary reaches the target size. The result: common words stay whole; rare words are split into frequent subword pieces.

```

# BPE conceptual walkthrough
# Corpus: ['low low low lower lower newest widest']

# Start: individual characters + end-of-word marker
# l o w </w> l o w e r </w> n e w e s t </w> w i d e s t </w>

# Count pairs: ('e','s') appears 2x → merge
# l o w </w> l o w e r </w> n e w e s t </w> w i d e s t </w>

# ('es','t') appears 2x → merge into 'est'
# l o w </w> l o w e r </w> n e w e s t </w> w i d e s t </w>

# After thousands of merges: vocab contains 'est', 'low', 'er', 'new', ...

from transformers import AutoTokenizer
gpt_tok = AutoTokenizer.from_pretrained('gpt2')

for w in ['transformer', 'unhappy', 'pneumonia', 'xyz123']:
    print(f'{w:15s} -> {gpt_tok.tokenize(w)}')
# transformer -> ['transform', 'er']
# unhappy      -> ['un', 'happy']
# pneumonia    -> ['pne', 'umonia']
# xyz123       -> ['xyz', '123']

```

Method 2: WordPiece — BERT, DistilBERT, RoBERTa

WordPiece is similar to BPE but merges based on likelihood rather than raw frequency. Its key identifier is the '##' prefix on continuation pieces. This marker means 'this piece continues the previous word — it is NOT a word start'.

```

from transformers import AutoTokenizer
bert_tok = AutoTokenizer.from_pretrained('bert-base-uncased')

for w in ['transformer', 'playing', 'unhappiness']:
    tokens = bert_tok.tokenize(w)
    print(f'{w:15s} -> {tokens}')
# transformer -> ['transform', '##er']
# playing      -> ['playing']          (common word, kept whole)
# unhappiness  -> ['un', '##happiness'] (or further split)

# ## means: this piece is NOT a word start
# Reconstructing words: join pieces, remove ##
tokens = bert_tok.tokenize('playing transformer music')
# ['playing', 'transform', '##er', 'music']
# → 'playing' | 'transform'+er='transformer' | 'music'

```

Method 3: SentencePiece — T5, Llama, Mistral

SentencePiece treats input as a raw byte stream — no pre-tokenization on spaces. This makes it language-agnostic and perfect for multilingual models. It marks word beginnings with a special underscore

character (U+2581: '▀') rather than marking continuations with '##'.

```
from transformers import AutoTokenizer
t5_tok = AutoTokenizer.from_pretrained('t5-small')

tokens = t5_tok.tokenize('Hello world from SentencePiece!')
print(tokens)
# ['▀Hello', '▀world', '▀from', '▀Sentence', 'P', 'iece', '!']
#  ↑ word start                ↑ continuation (no ▀)

# Vocabulary sizes:
# BERT (WordPiece):      30,522 tokens
# GPT-2 (BPE):           50,257 tokens
# T5 (SentencePiece):    32,100 tokens
# Llama 3 (SP+BPE):      128,256 tokens ← bigger vocab, fewer tokens per sequence
```

▀ Tokenizer Algorithm Comparison

Property	BPE (GPT-2)	WordPiece (BERT)	SentencePiece (T5)
Continuation marker	No marker	## prefix	▀ for word start
Language support	Good	Good	Excellent (any script)
Vocab size	~50k	~30k	~32k
Rare word handling	Byte fallback	[UNK] token	Byte fallback
Training algorithm	Frequency merge	Likelihood merge	Unigram language model
Space handling	Part of token (G)	Split at spaces	Encoded as ▀ prefix

Positional Encoding — Teaching Order to Attention

Attention is a permutation-invariant operation — 'The cat sat on the mat' and 'The mat sat on the cat' would produce identical attention scores if we only used token embeddings, because attention only looks at content, not position. Positional encodings fix this by injecting position information into every token's representation before the first attention layer.

■ Method 1: Sinusoidal Encoding (Original Paper)

The original 2017 paper uses sine and cosine waves of different frequencies. Even dimensions use sine, odd dimensions use cosine. Low-frequency waves capture coarse position; high-frequency waves capture fine position — like a binary counter where each bit has a different period.

```
# PE(pos, 2i)   = sin(pos / 10000^(2i/d_model))
# PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))

# Why sine and cosine?
# 1. Values always in [-1, 1] – won't overwhelm token embeddings
# 2. Nearby positions have similar encodings (smooth)
# 3. Can represent positions never seen during training (extrapolates)
# 4. PE(pos+k) can be written as a linear function of PE(pos)
#    → model can learn to attend to tokens k steps away

import torch, math

def sinusoidal_pe(max_len, d_model):
    pe = torch.zeros(max_len, d_model)
    pos = torch.arange(0, max_len).unsqueeze(1).float()
    div = torch.exp(torch.arange(0, d_model, 2).float() *
                    (-math.log(10000.0) / d_model))
    pe[:, 0::2] = torch.sin(pos * div) # even dims
    pe[:, 1::2] = torch.cos(pos * div) # odd  dims
    return pe # shape: [max_len, d_model]

pe = sinusoidal_pe(100, 128)
print(pe.shape) # [100, 128]
```

■ Method 2: Learned Positional Embeddings (BERT / GPT-2)

BERT and GPT-2 use a second embedding table indexed by position — the exact same mechanism as token embeddings, just indexed by [0, 1, 2, ..., seq_len-1] instead of token IDs. These are fully trained

alongside the rest of the model. The trade-off: simpler and often better in-distribution, but strictly limited to sequences no longer than the maximum seen during training (512 for BERT).

```
# BERT's position embeddings (from its config):
# position_embeddings = nn.Embedding(512, 768)
# → BERT CANNOT process sequences longer than 512 tokens

# To get around this on long documents:
# Option 1: Truncate to 512 tokens
# Option 2: Sliding window – process overlapping chunks
# Option 3: Use a model with longer context (Longformer, BigBird, Llama 3)

from transformers import AutoTokenizer
bert_tok = AutoTokenizer.from_pretrained('bert-base-uncased')
print('Max length:', bert_tok.model_max_length) # 512

# Test what happens with very long input:
long_text = 'word ' * 600
tokens = bert_tok(long_text, truncation=True, max_length=512)
print('Truncated to:', len(tokens['input_ids']), 'tokens')
```

■ Method 3: RoPE — Rotary Position Embedding (Modern LLMs)

RoPE, used by Llama, Mistral, Falcon, and most modern LLMs, encodes position by ROTATING the Query and Key vectors in complex space before computing attention. The beauty of RoPE: the dot product $Q \cdot K$ then depends naturally on the RELATIVE distance ($m-n$) between positions, not absolute positions. This gives better generalisation to sequences longer than seen during training.

```
# RoPE is applied inside the attention computation
# You do not implement it manually – it is built into LLM architectures

from transformers import AutoModelForCausalLM
model = AutoModelForCausalLM.from_pretrained('meta-llama/Llama-2-7b-hf')

# Context window sizes (check config):
print(model.config.max_position_embeddings) # Llama 2: 4096
# Mistral 7B: 8192 (with sliding window attention)
# Llama 3:    128,000 (with extended RoPE scaling)

# RoPE scaling tricks to extend context:
# - Linear scaling: compress position indices
# - YaRN (Yet Another RoPE Extension): change base frequency
# - LongRoPE: train-free extension to 2M+ tokens
```

■ Positional Encoding Methods Compared

Method	Models	Extrapolates?	Captures Relative Position?
Sinusoidal	Transformer (2017)	Partially	Not natively

Learned (abs.)	BERT, GPT-2	No — hard limit	No
ALiBi	BLOOM, MPT	Yes — very well	Via linear bias on scores
RoPE	Llama, Mistral	Yes (with tricks)	Yes — natively in Q/K dot

NLP Task Recipes — Copy-Paste Patterns

This chapter is your cookbook. Each section is a self-contained, runnable pattern for a specific NLP task. Every recipe follows the same structure: load model → tokenize → run → interpret output.

■ Recipe 1: Named Entity Recognition (NER)

NER identifies and classifies named entities: people, organisations, locations, dates. Unlike classification (one label per sentence), NER outputs a label for EVERY token. The standard label format is BIO: B=Begin, I=Inside, O=Outside entity.

```
from transformers import pipeline

ner = pipeline('ner',
               model='dslim/bert-base-NER',
               aggregation_strategy='simple') # merges sub-tokens

text = 'Elon Musk founded SpaceX in 2002 in El Segundo, California.'
for e in ner(text):
    print(f"{e['word']:20s} {e['entity_group']:5s} score={e['score']:.3f}")
# Elon Musk           PER    score=0.999
# SpaceX              ORG    score=0.998
# El Segundo          LOC    score=0.986
# California          LOC    score=0.994

# BIO labels explained:
# B-PER = Beginning of a Person entity
# I-PER = Inside a Person entity (continuation)
# O      = Outside any entity
# aggregation_strategy='simple' automatically merges B/I into one entity
```

■ Recipe 2: Extractive Question Answering

Extractive QA finds the answer span directly in the context passage. The model outputs start and end logits — scores for each token being the answer start/end. The answer is extracted from the passage, not generated from scratch.


```

from transformers import pipeline

qa = pipeline('question-answering',
              model='deepset/roberta-base-squad2')

context = '''
    The Transformer architecture was introduced by Vaswani et al. in 2017
    at Google Brain. It replaced recurrent networks with self-attention,
    enabling much better parallelism and setting the foundation for BERT,
    GPT, T5, and every modern LLM.
'''

questions = [
    'Who created the Transformer?',
    'What did the Transformer replace?',
    'What year was it introduced?',
]

for q in questions:
    result = qa(question=q, context=context)
    print(f'Q: {q}')
    print(f'A: {result["answer"]} (score={result["score"]:.3f})')
    print()

```

■ Recipe 3: Multilingual NLP with XLM-RoBERTa

XLM-RoBERTa is trained on 100 languages simultaneously. Fine-tuning it on English data gives you a model that works on all 100 languages — this is called zero-shot cross-lingual transfer, and it is one of the most powerful tricks in multilingual NLP.

```

from transformers import pipeline

# Multilingual sentiment – trained on multiple languages
clf = pipeline('sentiment-analysis',
              model='nlptown/bert-base-multilingual-uncased-sentiment')

texts = [
    ('English', 'This product is absolutely amazing!'),
    ('French', 'Ce produit est absolument incroyable!'),
    ('German', 'Dieses Produkt ist absolut erstaunlich!'),
    ('Spanish', 'Este producto es absolutamente increíble!'),
    ('Chinese', '■■■■■■■■■■'),
]

for lang, text in texts:
    result = clf(text)[0]
    print(f'{lang:10s}: {result["label"]} ({result["score"]:.2f})')

```

■ Zero-shot cross-lingual transfer works because XLM-RoBERTa learns a shared multilingual representation space — 'happy' in English and 'heureux' in French end up near each other in the embedding space. Fine-tune once, deploy globally.

Understanding What Transformers Actually Learn

Transformers are often called 'black boxes', but decades of interpretability research have given us genuine insights into what different layers and heads actually learn. This knowledge is not just academic — it directly informs practical choices like which layers to freeze, how to debug poor performance, and how to design better models.

■ Layer-by-Layer: What BERT Learns

Research using probing classifiers (simple linear models trained on each layer's output) has revealed a clear hierarchy in what BERT encodes at different depths:

BERT Layer	What It Primarily Encodes
Layer 0–1	Surface features: POS (noun/verb), morphology (plurals, tenses)
Layer 2–4	Syntactic structure: dependency parse, phrase boundaries
Layer 5–8	Semantic roles: subject, object, predicate relationships
Layer 9–11	High-level semantics: coreference, named entity types, sentence meaning
Layer 12	Task-specific signal: classification information at [CLS] token

■ Extracting and Comparing Layer Representations

You can extract representations from any layer using `output_hidden_states=True`. A classic experiment: take a polysemous word like 'bank' (financial vs river), encode it in two different contexts, and watch how the layers contextualise it.

```

from transformers import AutoTokenizer, AutoModel
import torch, torch.nn.functional as F

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model      = AutoModel.from_pretrained('bert-base-uncased',
                                       output_hidden_states=True)

def get_layer_reps(text, target_word):
    inputs  = tokenizer(text, return_tensors='pt')
    tokens  = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])
    with torch.no_grad():
        outputs = model(**inputs)
    idx = tokens.index(target_word)
    # Return vector for target_word at each of 13 layers
    return [outputs.hidden_states[l][0, idx, :]] for l in range(13)]

# Two sentences with 'bank' in different senses
financial = get_layer_reps('I deposited money at the bank', 'bank')
river_bank = get_layer_reps('The children played near the bank', 'bank')

# Compute similarity at each layer
for layer_idx in range(13):
    sim = F.cosine_similarity(financial[layer_idx],
                             river_bank[layer_idx], dim=0).item()
    print(f'Layer {layer_idx:2d}: similarity = {sim:.3f}')
# Layer 0:  similarity = 0.998  (almost identical – just the token embedding)
# Layer 6:  similarity = 0.87   (diverging – syntax context differs)
# Layer 12: similarity = 0.61   (very different – sense disambiguation!)

```

■ Known Attention Head Specialisations

Research by Clark et al. (2019) — 'What Does BERT Look At?' — found that specific attention heads reliably track linguistic relationships across different inputs:

Pattern	Layer Range	Description
Syntactic heads	2–5	Track dependency relations (subject attends to verb)
Positional heads	0–2	Strongly attend to the previous or next token
[CLS] aggregator	6–7	Gradually pulls information into the [CLS] token
Coreference	10–11	Links pronouns to their noun antecedents
[SEP] sink	All layers	Many heads dump excess attention onto [SEP]

■ The Residual Stream: A Useful Mental Model

A powerful way to think about transformers: every token has a 'residual stream' — a vector that flows unchanged through the network unless a layer chooses to modify it. Each layer READS from this stream and ADDS its contribution back (the residual connection). Nothing is ever erased — only additions happen.

This means:

- Information can skip layers unchanged via the residual path.
- Later layers can refine or overwrite information from earlier layers.
- The input token embedding is still partially present at the final layer.
- Removing a layer entirely causes less damage than you would expect — explaining why pruning works.

```
# Practical use: layer freezing for faster fine-tuning
# Lower layers (syntax, morphology) rarely need to change.
# Only train the top 3-4 layers for most classification tasks.

from transformers import AutoModel
model = AutoModel.from_pretrained('bert-base-uncased')

# Freeze everything except the last 3 transformer layers
for layer in model.encoder.layer[:9]: # freeze layers 0-8
    for param in layer.parameters():
        param.requires_grad = False

trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
total     = sum(p.numel() for p in model.parameters())
print(f'Trainable: {trainable:,} / {total:,} = {trainable/total:.1%}')
# Trainable: ~21M / 109M = 19.4%
# Training speed: ~5x faster with minimal accuracy loss on most tasks
```

■ *The layer hierarchy insight is directly useful: if your task is low-level (NER, POS tagging), fine-tuning the middle layers matters most. If your task is high-level semantics (sentiment, textual entailment), the top layers are where the action is.*

The Mental Model Every Beginner Needs

Most people learn transformers by memorising APIs. That works until something breaks or you need to make a non-standard choice. This chapter builds the intuition that makes everything else click: what is the transformer actually doing in plain language?

■ The Library Analogy

Imagine you are trying to understand the word 'bank' in a sentence. In 'I deposited money at the bank', the word 'money' tells you it is a financial bank. In 'The kids played by the bank of the river', 'river' tells you it is a riverbank. This is exactly what attention does — for every token, it asks: which other tokens in this sentence should I pay attention to in order to understand my meaning? It then combines weighted information from those tokens into a richer representation. After 12 layers of this, 'bank' means something completely different depending on context. That is contextualised embeddings.

■ The Assembly Line Analogy

Think of a transformer as an assembly line where each station refines the product:

Station	Output	What Happens
Input	Raw token IDs	Words converted to integers
Embedding Layer	Static 768-dim vectors	Each token ID → dense vector
+ Pos Encoding	Position-aware vectors	Vectors now know WHERE they appear
Layer 1–4	Syntax-enriched vectors	Local neighbour context captured
Layer 5–8	Semantic vectors	Long-range relationships resolved
Layer 9–12	Task-aware vectors	High-level meaning + task signal
Output Head	Prediction	Logits → label or next token

■ Why Pre-Training + Fine-Tuning Works So Well

When BERT trained on 3.3 billion words, it did not memorise facts — it learned representations of language: how words relate, what verb tenses mean, how pronouns refer back to nouns, how negation flips sentiment. When you fine-tune on your 10,000-example dataset, you are not teaching language from scratch. You are teaching one small extra thing: in your already-rich language representations, these patterns correlate with positive, and these with negative. That is why 3 epochs on 10k examples

generalises beautifully — 99 percent of the work was done during pre-training.

■ Encoder vs Decoder vs Encoder-Decoder — The Clear Rule

Type	Reads	Sees future?	Best For	Example Models
Encoder (BERT)	Full seq	Yes (both dirs)	Understanding: classify, NER, QA	BERT, RoBERTa, DistilBERT
Decoder (GPT)	Left-to-right	No (causal)	Generation: complete text	GPT-2, Llama, Mistral
Enc-Dec (T5)	Full input	Enc:yes Dec:no	Transform: translate, summarise	T5, BART, mBART

■ Rule of thumb: output is a LABEL or SPAN? Use encoder (BERT family). Output is FREE TEXT? Use decoder (GPT family) or encoder-decoder (T5/BART). When in doubt, T5's text-to-text framing handles almost any NLP task.

Hyperparameter Reference — Encoder vs Decoder

This chapter is your complete reference for every important hyperparameter in transformer models. For each parameter you get: what it controls, the recommended range, how encoder models (BERT family) and decoder models (GPT/Llama family) differ in their sensitivity to it, and what happens when you get it wrong.

■ ■ Part A — Architecture Hyperparameters

These are fixed at model creation time. If you use a pre-trained model you inherit these values. They only matter if you are building a custom model from scratch or selecting which pre-trained model to use.

d_model — Hidden Dimension Size

The width of the residual stream — the number of dimensions in every token's vector representation throughout the model. Every linear layer, attention projection, and feed-forward layer operates at this dimensionality. Larger d_model means more representational capacity but exponentially more parameters and memory.

Model Size	d_model	Encoder Examples	Decoder Examples
Tiny	128	Custom lightweight models	GPT-2 small variants
Small	256–384	DistilBERT (768/2=384)	GPT-Neo 125M
Base	768	BERT-base, RoBERTa-base	GPT-2 (768)
Large	1024	BERT-large, RoBERTa-large	GPT-2 Large
XL / 3B	2048	DeBERTa-xxlarge	GPT-NeoX 20B uses 6144
7B LLM	4096	N/A (encoder not used)	Llama 2/3 7B, Mistral 7B
13B+ LLM	5120+	N/A	Llama 2 13B (5120)

■ Rule: d_model must be divisible by num_heads (for multi-head attention). If d_model=768 and num_heads=12, each head gets d_model/num_heads = 64 dimensions.

num_layers (depth) — Number of Transformer Blocks

The number of stacked transformer blocks. More layers = more abstract representations, more capacity to model complex relationships, but also more parameters, slower training, and greater risk of vanishing gradients (mitigated by residual connections + LayerNorm). Depth typically matters more than width for

language tasks.

num_layers	Typical Model	Notes
2–4	Tiny custom models	Fast, low memory — for prototyping or edge deployment
6	DistilBERT (encoder)	Distilled from 12 layers, 40% faster with ~97% performance
12	BERT-base, GPT-2	Standard base model depth — good starting point
24	BERT-large, GPT-2 XL	Large — significantly more capable but 2x slower
32	Llama 2 7B (decoder)	Modern LLM standard depth for 7B parameter class
40	Llama 2 13B (decoder)	13B class
80	Llama 2 70B (decoder)	70B class — requires multi-GPU

num_heads — Number of Attention Heads

The number of parallel attention heads. Each head learns to attend to different aspects of the input simultaneously — one head might track syntactic dependencies, another coreference, another positional patterns. More heads = more diverse attention patterns, but each head gets fewer dimensions ($d_{\text{model}} / \text{num_heads}$).

num_heads	d_model	d_k per head	Model Examples
4	256	64	Small custom models
8	512	64	T5-small ($512/8=64$)
12	768	64	BERT-base, RoBERTa-base, GPT-2
16	1024	64	BERT-large, GPT-2 XL
32	4096	128	Llama 2 7B, Mistral 7B
40	5120	128	Llama 2 13B
64	8192	128	Llama 2 70B

d_ff — Feed-Forward Dimension

The hidden size of the two-layer MLP (feed-forward network) inside each transformer block. The FFN expands from d_{model} to d_{ff} , applies an activation, then contracts back. This is where most of the 'knowledge storage' is thought to happen. The standard ratio is $d_{\text{ff}} = 4 \times d_{\text{model}}$ (e.g. $768 \times 4 = 3072$ for BERT-base).

```

# Standard: d_ff = 4 * d_model
# BERT-base:      d_model=768, d_ff=3072      (ratio 4x)
# GPT-2:         d_model=768, d_ff=3072      (ratio 4x)
# Llama 2 7B:     d_model=4096, d_ff=11008    (ratio ~2.7x — uses SwiGLU activation)
# Mistral 7B:     d_model=4096, d_ff=14336    (ratio 3.5x — uses SwiGLU)

# SwiGLU (used by Llama/Mistral) uses 2/3 ratio to compensate:
# SwiGLU FFN:  $x \rightarrow (\text{gate} * \text{SiLU}(W_1 * x)) * W_2$ 
# Uses 3 matrices instead of 2, so d_ff is reduced to keep params equal

# In custom model config:
from transformers import BertConfig
config = BertConfig(
    hidden_size=768,          # d_model
    intermediate_size=3072,   # d_ff (= 4 * hidden_size)
    num_hidden_layers=12,     # num_layers
    num_attention_heads=12,   # num_heads
)

```

max_position_embeddings — Context Window

The maximum number of tokens the model can process at once. Exceeding this limit causes an error for absolute position models (BERT), or degraded output for RoPE models (Llama). This is one of the most practically important architecture choices — longer context = more memory quadratically (attention is $O(n^2)$ in sequence length).

Context Window	Memory Cost	Model Examples
512 tokens	Low	BERT-base, DistilBERT — ~380 words
1,024 tokens	Low	GPT-2 — ~770 words
2,048 tokens	Moderate	GPT-NeoX, early Llama — ~1,500 words
4,096 tokens	Moderate	Llama 2, Mistral 7B — ~3,000 words
8,192 tokens	High	Mistral (sliding window), CodeLlama
32,768 tokens	Very high	Llama 3, extended models — ~25,000 words
128,000 tokens	Extreme	Llama 3.1, GPT-4 — ~96,000 words

■ ■ Memory for attention scales as $O(n^2)$. Doubling the context window quadruples attention memory. This is why models with 128k context need specialised hardware or techniques like FlashAttention and sliding window attention.

■ ■ Part B — Training Hyperparameters

These are the knobs you turn during fine-tuning. This is where most of your experimentation happens. Encoder models (BERT) and decoder models (GPT/Llama) have meaningfully different sweet spots for many of these — the table below makes this explicit for every parameter.

learning_rate

Controls the step size during gradient descent. The single most important training hyperparameter. Too high: weights overshoot, loss oscillates or diverges. Too low: training is very slow, may get stuck in local minima. Decoders typically need a higher LR than encoders for the same task because their larger capacity means the gradient signal is more spread out.

Scenario	Encoder (BERT family)	Decoder (GPT/Llama family)
Fine-tuning (standard)	1e-5 to 5e-5	5e-6 to 2e-5
Fine-tuning (small dataset <2k)	1e-5 to 2e-5	5e-6 to 1e-5
Fine-tuning with LoRA adapters	1e-4 to 5e-4	1e-4 to 3e-4
Training from scratch	1e-4 to 1e-3	1e-4 to 6e-4
QLoRA (4-bit fine-tuning)	N/A	2e-4 to 5e-4
Sweet spot default	2e-5	1e-5
Signs it is too HIGH	Loss spikes / NaN	Loss spikes / NaN
Signs it is too LOW	Loss barely moves	Loss barely moves

batch_size (per_device_train_batch_size)

Number of training examples processed together in one forward + backward pass. Larger batches give more accurate gradient estimates (less noise) but use more GPU memory and can hurt generalisation (the 'sharp minima' effect). Encoder models fine-tune well with larger batches; decoder generation tasks typically use smaller batches due to memory constraints from long sequences.

Scenario	Encoder (BERT family)	Decoder (GPT/Llama family)
Fine-tuning classification	16–32	4–16 (long inputs = more memory)
Fine-tuning NER / token cls	16–32	N/A (not typical for decoders)
Fine-tuning QA	8–16	4–8
Instruction fine-tuning LLM	N/A	1–8 (QLoRA on 24GB GPU: 2–4)
Training from scratch	256–2048	256–4096 (GPT-3 used 3.2M!)
Memory constraint workaround	Use gradient_accum_steps	Use gradient_accum_steps

num_train_epochs

Total number of passes through the training dataset. For fine-tuning, transformers almost always need fewer epochs than you expect — the pre-trained weights are already very close to the solution. More epochs = higher risk of overfitting and 'catastrophic forgetting' of pre-training knowledge. Use early stopping to let the data tell you when to stop.

Task	Encoder (BERT family)	Decoder (GPT/Llama family)
Text classification (large data)	2-3	N/A
Text classification (small data)	5-10	N/A
Named Entity Recognition	3-5	N/A
Extractive QA	2-4	N/A
Instruction fine-tuning	N/A	1-3
Chat / RLHF fine-tuning	N/A	1-2
Continued pre-training	1-3	1-3
Signs of too many epochs	Val loss rising	Val loss rising / repetition

warmup_steps / warmup_ratio

Gradually ramps the learning rate from 0 to the target LR over the first N steps. This prevents early instability: at step 0 the model weights are the same as the pre-trained checkpoint, so large gradient steps would destroy useful representations. The warmup period lets the new task-specific head initialise before the LR reaches full power.

```
# warmup_ratio is cleaner than warmup_steps (adapts to dataset size)
# warmup_steps = warmup_ratio × total_training_steps

# Encoder models (BERT fine-tuning):  warmup_ratio = 0.06  (6% of steps)
# Decoder LLM fine-tuning:             warmup_ratio = 0.03  (3% of steps)
# Training from scratch:               warmup_ratio = 0.01-0.04

training_args = TrainingArguments(
    warmup_ratio=0.06,      # OR
    # warmup_steps=500,    # fixed number of warmup steps
)

# Manual: linear warmup then cosine decay (common in LLM training)
from transformers import get_cosine_schedule_with_warmup
total_steps = len(train_loader) * num_epochs
warmup_steps = int(0.06 * total_steps)
scheduler = get_cosine_schedule_with_warmup(
    optimizer, num_warmup_steps=warmup_steps,
    num_training_steps=total_steps,
)
```

weight_decay

L2 regularisation — adds a penalty proportional to the square of each weight to the loss function. This encourages smaller weights and reduces overfitting. Applied to all parameters EXCEPT bias and LayerNorm (HuggingFace Trainer does this correctly by default). Too high: underfitting. Too low: overfitting on small datasets.

weight_decay	Encoder (BERT family)	Decoder (GPT/Llama)
0.0	No regularisation — risky on small data	No regularisation
0.01	Standard default — works well most times	Standard for SFT
0.05–0.1	Strong — use on small datasets (< 2k)	Aggressive — rarely needed
0.1+	Very strong — usually causes underfitting	Too strong

dropout / attention_dropout

During training, randomly zeroes a fraction of neurons in the feed-forward layers (`hidden_dropout_prob`) and in the attention weight matrix (`attention_probs_dropout_prob`). Forces the model to not rely on any single neuron, improving generalisation. Set to 0.0 during inference (`model.eval()` does this automatically). Decoders tend to use lower dropout than encoders because they have more parameters and generative tasks require more coherent representations.

dropout value	Effect	Encoder typical	Decoder typical
0.0	No dropout (fast, overfit)	From-scratch training	LLM fine-tuning
0.1	Light regularisation	Standard (BERT default)	Standard GPT
0.2	Moderate	Small datasets	Rarely used
0.3–0.5	Heavy	Very small data only	Avoid for decoders

max_length / max_seq_length

The maximum number of tokens per training example. Sequences longer than this are truncated; shorter ones are padded. Longer `max_length` = more context but quadratically more memory for attention ($O(n^2)$). For classification tasks, most signal is in the first 512 tokens. For generation, longer context is more critical.

```

# Encoder models – typical max_length values:
# Classification:          128-256  (most sentiment fits in 128 tokens)
# NER:                     128-256  (named entities are in local context)
# Extractive QA:           384-512  (passage can be long)
# Document classification:  512      (BERT hard limit)

# Decoder models – typical max_length values:
# Short instruction following: 512-1024
# Long-form generation:       2048-4096
# Summarisation (input):      1024-4096
# Code generation:            2048-8192

# Memory tip: halving max_length reduces attention memory by ~4x
# Try 128 first for classification – usually loses < 1% accuracy vs 512
tokenizer(text, truncation=True, max_length=128) # for fast iteration

```

gradient_clip_norm (max_grad_norm)

Clips the norm of the gradient vector to a maximum value before the weight update. Prevents 'exploding gradients' — when a single bad batch causes an enormous gradient step that destroys the model weights. Almost always set to 1.0. If you see NaN loss, this is usually the first thing to check.

```

# Via TrainingArguments (default is 1.0 – always keep this on):
training_args = TrainingArguments(
    ...
    max_grad_norm=1.0,  # clip gradient norm to 1.0
)

# Manual (if writing your own training loop):
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()

# Signs you need gradient clipping:
# - Loss suddenly jumps to NaN
# - Loss was stable then suddenly explodes
# - gradient norm log shows values > 10

# Encoder: max_grad_norm=1.0 always sufficient
# Decoder LLMs: max_grad_norm=1.0 standard; sometimes 0.5 for stability

```

■ ■ Part C — Optimiser Hyperparameters

These control how the gradients are used to update the weights. AdamW is the standard for transformer fine-tuning. Understanding its parameters helps you debug convergence issues.

AdamW — beta1, beta2, epsilon

AdamW maintains two running averages per parameter: the first moment (momentum, controlled by beta1) and the second moment (variance, controlled by beta2). These adapt the effective learning rate for each parameter individually. The defaults work for almost all cases — only change if you have a specific reason.

Parameter	Default	What It Controls	When to Change
beta1	0.9	Momentum — smooths gradient direction	Rarely: try 0.95 for noisy gradients
beta2	0.999	Variance tracking — adapts per-param LR	LLMs often use 0.95 for stability
epsilon	1e-8	Numerical stability (avoids div by zero)	fp16: use 1e-6 to avoid underflow
weight_decay	0.01	L2 penalty (only in AdamW, not Adam)	0.01–0.1 range

```
from torch.optim import AdamW

# Standard encoder fine-tuning (BERT):
optimizer = AdamW(
    model.parameters(),
    lr=2e-5,
    betas=(0.9, 0.999),
    eps=1e-8,
    weight_decay=0.01,
)

# LLM fine-tuning (Llama / Mistral) – common adjustments:
optimizer = AdamW(
    model.parameters(),
    lr=2e-4,           # higher for LoRA
    betas=(0.9, 0.95), # lower beta2 for stability with large models
    eps=1e-5,         # larger epsilon for fp16/bf16
    weight_decay=0.01,
)

# HuggingFace Trainer handles the optimizer for you by default.
# Override with optimizers= argument if you need custom settings.
```

LR Schedule — Which Decay Strategy to Use

After the warmup phase, the learning rate should decrease so the model makes progressively finer adjustments. The choice of decay curve matters more for long training runs than for short fine-tuning jobs.

Schedule	Shape	Best For	HF Name
----------	-------	----------	---------

Linear decay	Straight drop	Standard BERT fine-tuning	linear
Cosine decay	S-curve drop	Most tasks — smooth landing	cosine
Constant	Flat	Quick experiments only	constant
Cosine w/ restarts	Cycles	Long pre-training runs	cosine_with_restarts
Polynomial	Power law drop	Custom control over decay	polynomial
Inverse sqrt	1/sqrt(step)	Original Transformer paper	inverse_sqrt

```
# Use via TrainingArguments:
training_args = TrainingArguments(
    lr_scheduler_type='cosine',  # linear | cosine | constant | ...
    warmup_ratio=0.06,
)

# Or manually:
from transformers import get_cosine_schedule_with_warmup
scheduler = get_cosine_schedule_with_warmup(
    optimizer,
    num_warmup_steps=warmup_steps,
    num_training_steps=total_steps,
)
```


■ Part D — LoRA / PEFT Hyperparameters

LoRA introduces its own set of hyperparameters. These are especially important for decoder LLM fine-tuning where LoRA is the dominant approach. Getting these right can make the difference between a model that follows instructions and one that just repeats the prompt.

Parameter	What It Controls	Encoder (BERT)	Decoder (LLM)
r (rank)	Size of adapter matrices. Higher = more capacity, but more params.	4–16	8–64
lora_alpha	Scaling factor: effective_LR = alpha/r. Usually set to 2 * r.	16–32	16–64
lora_dropout	Dropout on adapter outputs. Reduces overfitting	0.05–0.1	0.05–0.1
target_modules	Which weight matrices to add adapters to. More [query, value].	['query', 'value']	['q_proj', 'v_proj', 'k_proj', 'o_proj']
bias	Whether to train bias terms alongside adapters. 'none'	'none'	'none' or 'all'
task_type	Tells PEFT what kind of task this is (affects how sequences are tokenized)	SEQ_CLS or SEQ_LST	CAUSAL_LM

```
from peft import LoraConfig, TaskType

# Encoder (BERT classification) – conservative LoRA
encoder_lora = LoraConfig(
    task_type=TaskType.SEQ_CLS,
    r=8,                # small rank – BERT is already well-trained
    lora_alpha=16,      # alpha = 2 * r
    target_modules=['query', 'value'],
    lora_dropout=0.1,
    bias='none',
)

# Decoder LLM (Llama/Mistral instruction fine-tuning) – richer LoRA
decoder_lora = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16,               # higher rank for complex instruction following
    lora_alpha=32,      # alpha = 2 * r
    target_modules=['q_proj', 'v_proj', 'k_proj', 'o_proj',
                   'gate_proj', 'up_proj', 'down_proj'], # include FFN
    lora_dropout=0.05,
    bias='none',
)
```

■ Part E — Inference & Generation Hyperparameters

These only apply to decoder models during text generation. They have no equivalent in encoder models. The right combination depends entirely on your use case — there is no universal 'best' setting.

Parameter	What It Controls	Range	Recommended Value
max_new_tokens	Max tokens the model generates	1 – context limit	Task-dependent (50–2048)

temperature	Sharpens (< 1) or flattens (> 1) the probability distribution.	0.1–1.0	0.7 (chat) 0.1 (factual)
top_k	Only sample from the top-k most likely tokens.	1 – vocab_size	50 (default) 1 = greedy
top_p (nucleus)	Sample tokens covering top p% of probability mass.	0.1–1.0	0.9 (creative) 1.0 = off
repetition_penalty	Penalise tokens already generated. > 1 = reduce repeats.	1.0–1.5	1.1–1.3 (long generation)
num_beams	Beam search: keep N candidate sequences.	1–10 greedy.	4–5 (summarisation/translation)
do_sample	True = sample, False = greedy/beam.	True / False	True for creative, False for factual
no_repeat_ngram_size	Blocks any n-gram from appearing twice.	0 – 4	2–3 (beam search)
penalty_alpha	Contrastive search strength. 0 = greedy, higher = more diverse.	0.0 – 1.0	0.6 (contrastive search)

■ Quick-Reference: The Complete Defaults Card

Copy this block as your starting point for any new fine-tuning job and adjust from there.

Encoder (BERT/RoBERTa/DistilBERT) — Classification / NER / QA

```

training_args = TrainingArguments(
    output_dir          = './output',
    num_train_epochs    = 3,
    per_device_train_batch_size = 16,
    per_device_eval_batch_size = 32,
    learning_rate       = 2e-5,           # range: 1e-5 to 5e-5
    warmup_ratio        = 0.06,          # 6% warmup
    weight_decay        = 0.01,          # L2 regularisation
    lr_scheduler_type   = 'linear',       # or 'cosine'
    max_grad_norm       = 1.0,           # gradient clipping
    evaluation_strategy = 'epoch',
    save_strategy       = 'epoch',
    load_best_model_at_end = True,
    fp16               = True,           # GPU only
    seed               = 42,
    report_to          = 'none',
)

```

Decoder LLM (Llama/Mistral) — Instruction Fine-tuning with LoRA

```

training_args = TrainingArguments(
    output_dir          = './llm-output',
    num_train_epochs    = 2,
    per_device_train_batch_size = 4,
    gradient_accumulation_steps = 8,      # effective batch = 32
    learning_rate       = 2e-4,          # higher for LoRA
    warmup_ratio        = 0.03,          # 3% warmup
    weight_decay        = 0.01,
    lr_scheduler_type   = 'cosine',
    max_grad_norm       = 1.0,
    optim               = 'paged_adamw_32bit', # memory efficient
    bf16               = True,             # Ampere GPU+; else fp16=True
    max_steps           = -1,              # -1 = use num_train_epochs
    evaluation_strategy = 'steps',
    eval_steps          = 200,
    save_steps          = 200,
    load_best_model_at_end = True,
    seed               = 42,
    report_to           = 'none',
)

```

■ *These defaults get you 80% of the way on any new task. The most impactful things to tune, in order: `learning_rate` → `num_epochs` → `batch_size`. Everything else is secondary unless you have a specific problem to fix.*

Hyperparameter Tuning — What to Change and Why

One of the most practical skills is knowing which knob to turn when your model is not performing well. This chapter gives you a systematic checklist and explains the reasoning behind each hyperparameter so you can diagnose your own problems.

■ The Tuning Checklist (Work Top to Bottom)

Step 1: Loss not decreasing at all?

Action: LR too high (try $1e-5$) or label bug. Check `compute_metrics` and label column name.

Step 2: Loss decreasing then plateauing?

Action: Underfitting. Try: more epochs, higher LR, larger model, unfreeze more layers.

Step 3: Train loss low, val loss high?

Action: Overfitting. Try: `weight_decay=0.01`, more dropout, fewer epochs, more data.

Step 4: Loss NaN or exploding?

Action: LR too high or no gradient clipping. Add `clip_grad_norm_(params, 1.0)`.

Step 5: Training very slow?

Action: Enable `fp16=True`, increase batch size + gradient accumulation, add `num_workers`.

Step 6: CUDA out of memory?

Action: Reduce batch size, enable gradient checkpointing, use `fp16`, reduce `max_length`.

Step 7: Not generalising to new data?

Action: Distribution shift. Check: does your fine-tune data match the real-world inputs?

■■ Learning Rate — The Most Critical Hyperparameter

Too high: weights overshoot, loss diverges. Too low: training is glacial. The sweet spot for fine-tuning is almost always between $1e-5$ and $5e-5$. For LoRA fine-tuning of LLMs, use a higher rate ($1e-4$ to $3e-4$) because the LoRA matrices are randomly initialised from scratch.

```
# Fine-tuning BERT/roBERTa:          2e-5 (try 1e-5 to 5e-5)
# Fine-tuning with LoRA (LLMs):      2e-4 (try 1e-4 to 3e-4)
# Training from scratch:             1e-3 (no pre-training to protect)

# Empirical LR search – train 1 epoch at each rate, pick lowest loss:
for lr in [1e-6, 5e-6, 1e-5, 3e-5, 1e-4, 5e-4]:
    args = TrainingArguments(learning_rate=lr, num_train_epochs=1, ...)
    result = Trainer(model=load_model(), args=args, ...).train()
    print(f'LR {lr:.0e}: loss = {result.training_loss:.4f}')
```

■ Epochs — How Long to Fine-tune

```
# Typical epoch counts:
# Text classification (50k+ examples): 2-3 epochs
# Text classification (< 5k examples): 5-10 epochs
# NER: 3-5 epochs
# Question Answering: 2-3 epochs
# LLM instruction fine-tuning (LoRA): 1-3 epochs

# Always use early stopping – stop when val loss stops improving:
from transformers import EarlyStoppingCallback
trainer = Trainer(
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2)],
    # requires: evaluation_strategy='epoch', load_best_model_at_end=True
)
```

■ *The most common beginner mistake: changing too many things at once. Change ONE hyperparameter per run. Keep a simple log: LR, batch, epochs, val F1. After 5-6 runs you will understand your task's sensitivity to each parameter.*

Data — More Important Than the Model

The transformer architecture and fine-tuning recipe are now commoditised. What differentiates great models from mediocre ones is almost always the quality and preparation of the data. This chapter covers everything you need.

■ Data Quality Checklist — Run This Before Every Training Job

```
import pandas as pd
df = dataset.to_pandas()

# 1. Missing values
print(df.isnull().sum())
df = df.dropna(subset=['text', 'label'])

# 2. Label distribution — is it balanced?
print(df['label'].value_counts(normalize=True))

# 3. Text length distribution — will many be truncated?
df['length'] = df['text'].str.split().str.len()
print(df['length'].describe())
print(f'Texts > 512 words: {(df.length > 512).mean():.1%}')

# 4. Duplicates — train/test leakage is a silent disaster
duplicates = df.duplicated(subset=['text'])
print(f'Duplicates: {duplicates.sum()} ({duplicates.mean():.1%}%)')
df = df.drop_duplicates(subset=['text'])

# 5. Spot-check labels — sample 5 per class, read them
for label in df['label'].unique():
    print(f'\n=== Label {label} ===')
    for text in df[df['label']==label]['text'].sample(3):
        print(' ', text[:120])
```

■■ Handling Class Imbalance

```

# Method 1: Weighted loss – penalise errors on rare classes more
import torch
weights = torch.tensor([1.0, 9.0]) # class 0 rare, class 1 common
criterion = torch.nn.CrossEntropyLoss(weight=weights.to(device))

# Method 2: Oversample minority class
from datasets import concatenate_datasets
minority = dataset.filter(lambda x: x['label'] == 0)
majority = dataset.filter(lambda x: x['label'] == 1)
ratio = len(majority) // len(minority)
balanced = concatenate_datasets([majority] + [minority]*ratio).shuffle(seed=42)

# Method 3: Always report macro-F1, not accuracy
# Accuracy is misleading on imbalanced data.
# A model always predicting majority class gets 90% accuracy on a 90/10 split.
f1 = evaluate.load('f1')
result = f1.compute(predictions=preds, references=labels, average='macro')

```

■ How Much Data Do You Actually Need?

Dataset Size	Expected Result	Recommendation
< 100 examples	Unreliable, high variance	Use few-shot prompting instead
100–1,000	Possible with small models + LoRA	DistilBERT, max regularisation
1k–10k	Good results on focused tasks	Standard fine-tuning + augmentation
10k–100k	Solid, production-ready	Standard recipe, experiment freely
> 100k	Excellent — diminishing returns above 1M	Consider full fine-tune from scratch

Mistakes I Wish Someone Had Told Me

These are the non-obvious mistakes that waste weeks of learning time. None of them are in the official docs. Each is the kind of thing you only discover by hitting a wall and spending hours confused.

1. Mismatched tokenizer and model

Every model has its own vocabulary. BERT's token ID 7592 means 'hello'. GPT-2's token ID 7592 means something completely different. Always load the tokenizer from the SAME checkpoint as the model.

```
# WRONG – different vocab, garbage output
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model     = AutoModelForCausalLM.from_pretrained('gpt2')

# CORRECT – always same checkpoint
tokenizer = AutoTokenizer.from_pretrained('gpt2')
model     = AutoModelForCausalLM.from_pretrained('gpt2')
```

2. Forgetting model.eval() during inference

In training mode, Dropout randomly zeros neurons — predictions are non-deterministic. model.eval() disables Dropout and switches BatchNorm to running stats. Always set eval mode before inference.

```
# WRONG – Dropout active, different result every call
output = model(**inputs)

# CORRECT
model.eval()
with torch.no_grad():      # also saves memory
    output = model(**inputs)
model.train()              # re-enable for next training step
```

3. Evaluating on the training set

Training accuracy is always inflated — the model has seen those examples. Always use a held-out split. Train/val/test contamination is the most common source of falsely good results in ML research.

```
# If no pre-split, create one before ANYTHING else:
splits      = dataset['train'].train_test_split(test_size=0.2, seed=42)
train_data  = splits['train']
eval_data   = splits['test']    # never used during training

# Seed matters: same seed = same split every run = reproducible
```


4. Not setting random seeds

Without seeds, every run gives different results. You cannot reproduce your best run, and you cannot tell if a change helped or was just lucky initialisation.

```
import torch, random, numpy as np

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(42)  # call this at the very top of every script
# TrainingArguments(seed=42) handles it automatically for Trainer
```

5. Not verifying tokenized data before training

Tokenization bugs are silent — the model trains but learns nothing meaningful. Always decode a few samples back to text and visually confirm they look correct.

```
# Always run this before starting a training job:
for i in range(3):
    ids = tokenized['train'][i]['input_ids']
    text = tokenizer.decode(ids, skip_special_tokens=False)
    label = tokenized['train'][i]['labels']
    print(f'--- Example {i} ---')
    print(f'Decoded: {text[:200]}')
    print(f'Label: {label}')
# Garbled text = bug in tokenize_function
# Missing [CLS]/[SEP] = wrong tokenizer settings
```

■ Every experienced practitioner has made all five of these mistakes — usually more than once. Knowing them in advance saves days of frustration. Bookmark this chapter.

Your First 30 Days: A Practical Action Plan

Theory without practice is forgotten in a week. This is a concrete day-by-day plan to take you from reading this guide to confidently building transformer systems. Each task is small enough to complete in one sitting, and each builds on the last.

■ Week 1 — Run Everything, Understand Later

The goal this week is NOT to understand everything. The goal is to get code running and see real results. Understanding comes from experimenting with working code.

Day 1: Setup

Install PyTorch + transformers. Run Chapter 2 verification. Check GPU is detected.

Day 2: Pipelines

Run every pipeline from Chapter 6 on your own text. Try sentiment on 5 sentences you write. See what the model gets wrong — it will surprise you.

Day 3: Tokenizers

Tokenize 10 sentences. Print the tokens. Find a word that splits into multiple pieces. Understand what [CLS], [SEP], [PAD] look like.

Day 4: Model outputs

Load BERT with AutoModel. Pass a sentence through. Print `last_hidden_state.shape`. Understand what [batch, seq_len, 768] means.

Day 5: Datasets

Load IMDB. Print 5 examples. Tokenize it. Decode a tokenized example back to text. Verify it looks correct.

Day 6-7: First fine-tune

Run the Chapter 8 fine-tuning example on IMDB. Watch the loss curve. Then predict 3 sentences you write yourself. Celebrate — you just built an NLP model.

■ Week 2 — Go Deeper, Break Things

Day 8-9: Experiment

Change your IMDB training: try $\text{lr}=1\text{e-}4$ (too high?), $1\text{e-}6$ (too slow?). Try 1 epoch vs 5. Log everything. Build intuition.

Day 10-11: New task

Pick NER or QA. Fine-tune a model from the Hub on a new dataset. Notice how the same pattern applies.

Day 12: Generation

Load GPT-2. Try all 6 decoding strategies. Compare outputs at temperature 0.3 vs 1.5. Build strong intuition for what each does.

Day 13: Embeddings

Install sentence-transformers. Encode 20 sentences. Compute all pairwise similarities. Find surprising pairs — semantically similar sentences that look different.

Day 14: Deploy

Build a Gradio demo for your sentiment model. Share the public link. Get real users to try it and give feedback.

■ Week 3-4 — Build Something Real

Stop following tutorials. Pick a real problem you actually care about. The constraint of a real goal exposes gaps in your knowledge faster than any tutorial.

Domain	Project Idea
General NLP	Classify support tickets by topic. Extract product names from reviews. Build an FAQ answering system.
Code	Classify bug vs non-bug commit messages. Score code comment quality.
Science	Fine-tune BioBERT on medical abstracts. Build a symptom-to-condition classifier.
Personal	Analyse your emails or messages for sentiment trends over time.
Multilingual	Build a language detector. Fine-tune XLM-R on a non-English dataset.

■ You Have Reached Intermediate When You Can...

- Load any Hub model and run inference in under 5 minutes — without Googling.
- Fine-tune a classifier, evaluate it properly, and justify the metrics you chose.
- Diagnose a bad training run (loss not moving, NaN, overfitting) without guessing.
- Choose encoder vs decoder vs encoder-decoder given a task description.
- Explain attention to a non-technical person using an analogy.
- Read a model card and decide whether a model fits your use case.

- ■ Build and deploy a working demo of a fine-tuned model.

■ *The fastest path to intermediate is building, breaking, and fixing real things — not reading more tutorials. After 30 days of active building you will know more than most people who have been passively learning for months. Go build something today.*